

Documentación de Modelos de IA

Autores: Anthony Jiménez, Aaron Gónzales

Datasets utilizados:

BodyFat: <https://www.kaggle.com/datasets/fedesoriano/body-fat-prediction-dataset>

Bitcoin: <https://www.kaggle.com/datasets/team-ai/bitcoin-price-prediction/version/1>

Vino: <https://www.kaggle.com/datasets/rajyellow46/wine-quality>

Aguacate: <https://www.kaggle.com/datasets/neuromusic/avocado-prices>

Telecom Churn: <https://github.com/IBM/telco-customer-churn-on-icp4d/blob/master/data/Telco-Customer-Churn.csv>

Modelo: Aguacate

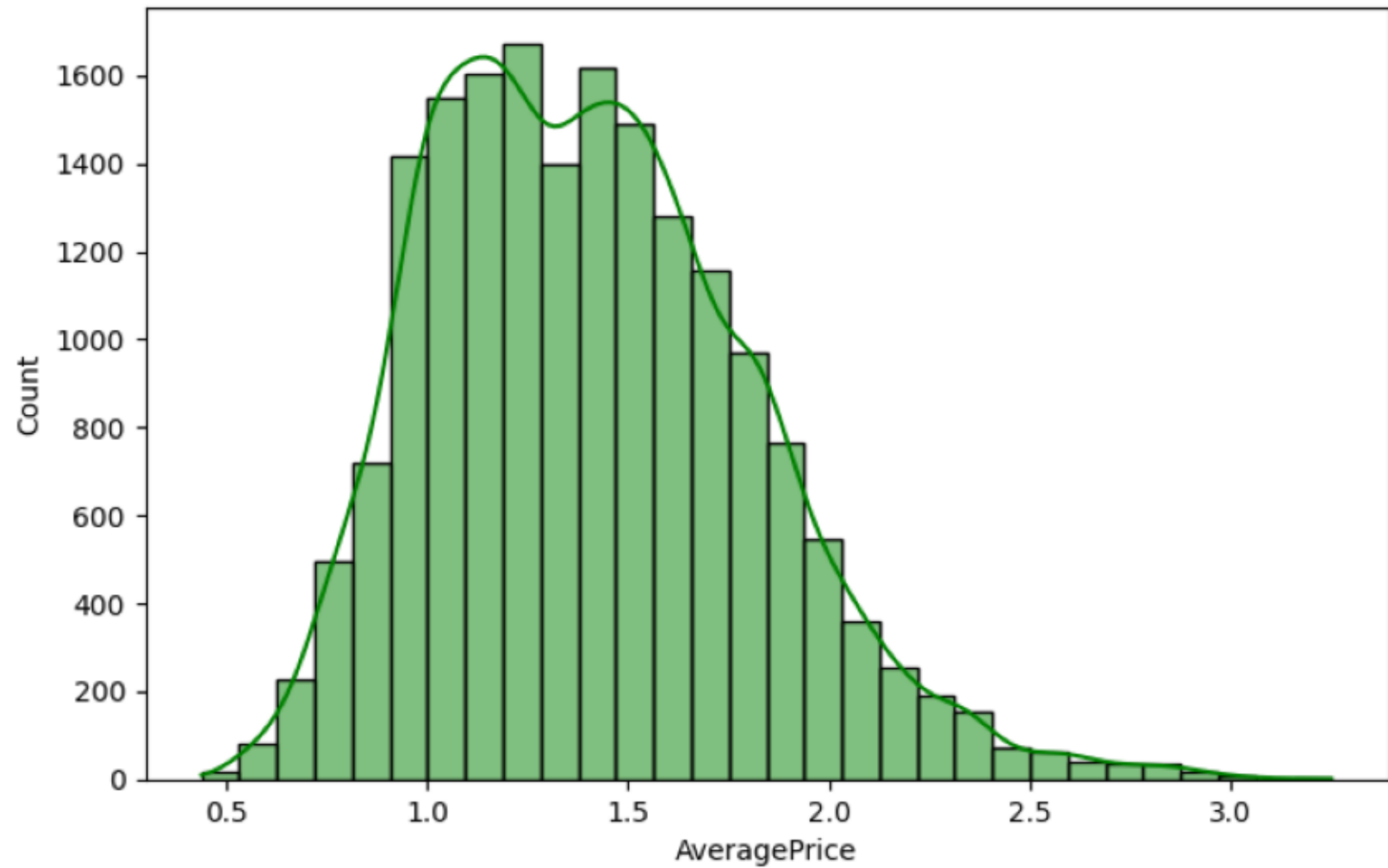
Se predice el precio promedio (AveragePrice) de los aguacates según volumen de venta, tipo, año y región. Modelo: Random Forest Regressor.

Métricas del modelo Aguacate

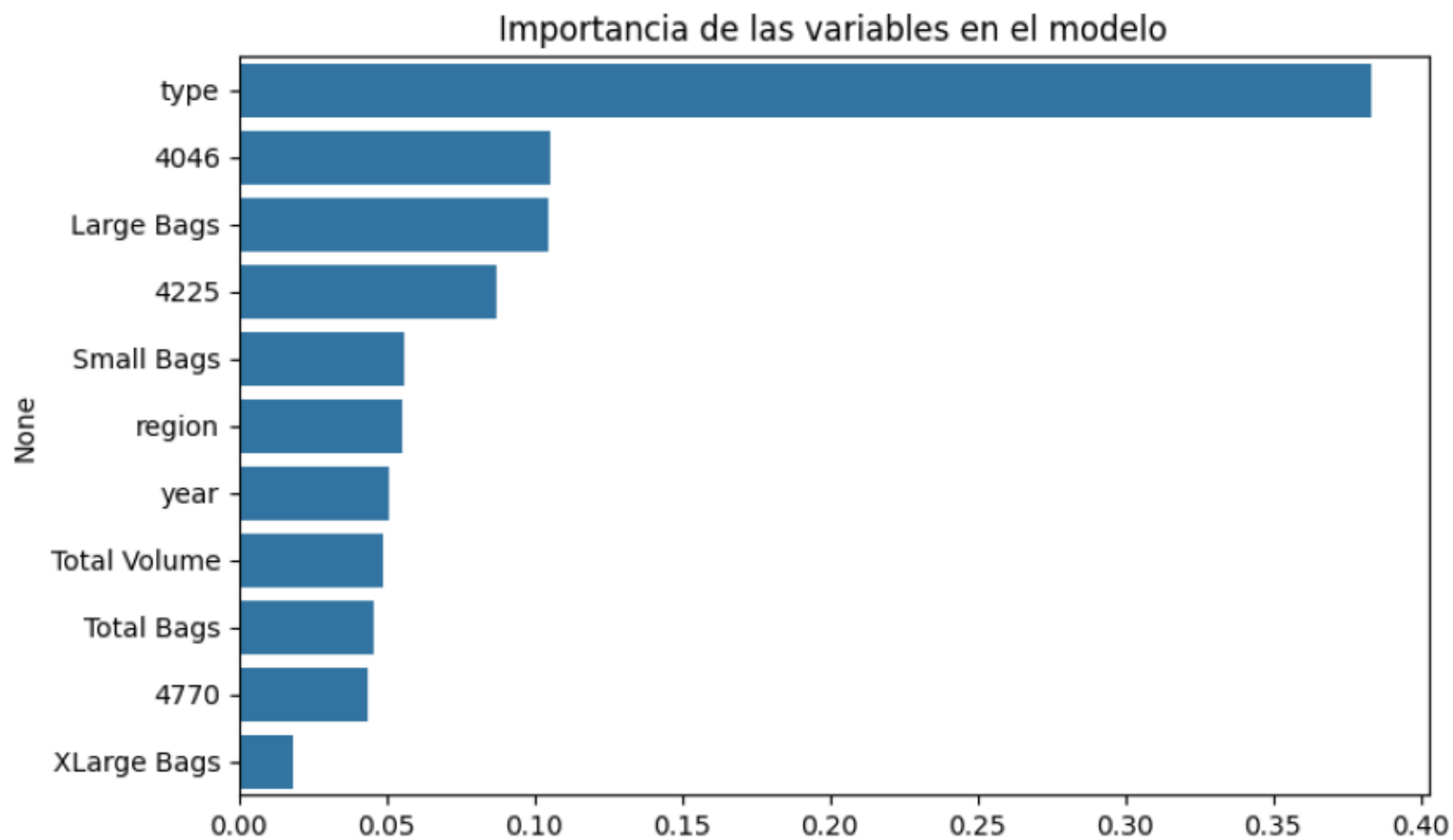
Métrica	Valor
MAE	0.1097624931506849
RMSE	0.1557360116852118
R^2	0.8490455983516202

Aguacate - averageprice_distribution.png

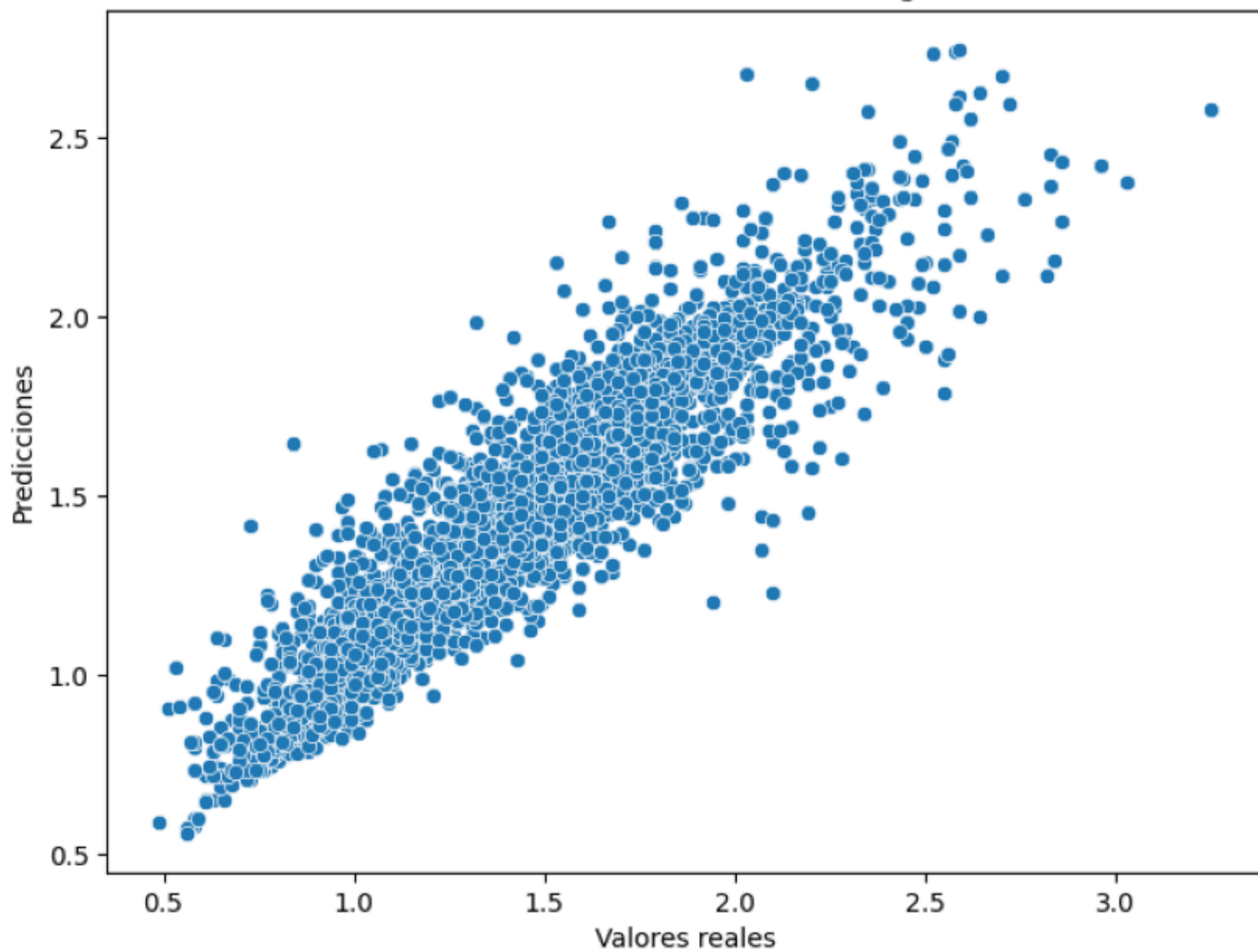
Distribución del Precio Promedio del Aguacate



Aguacate - feature_importance.png

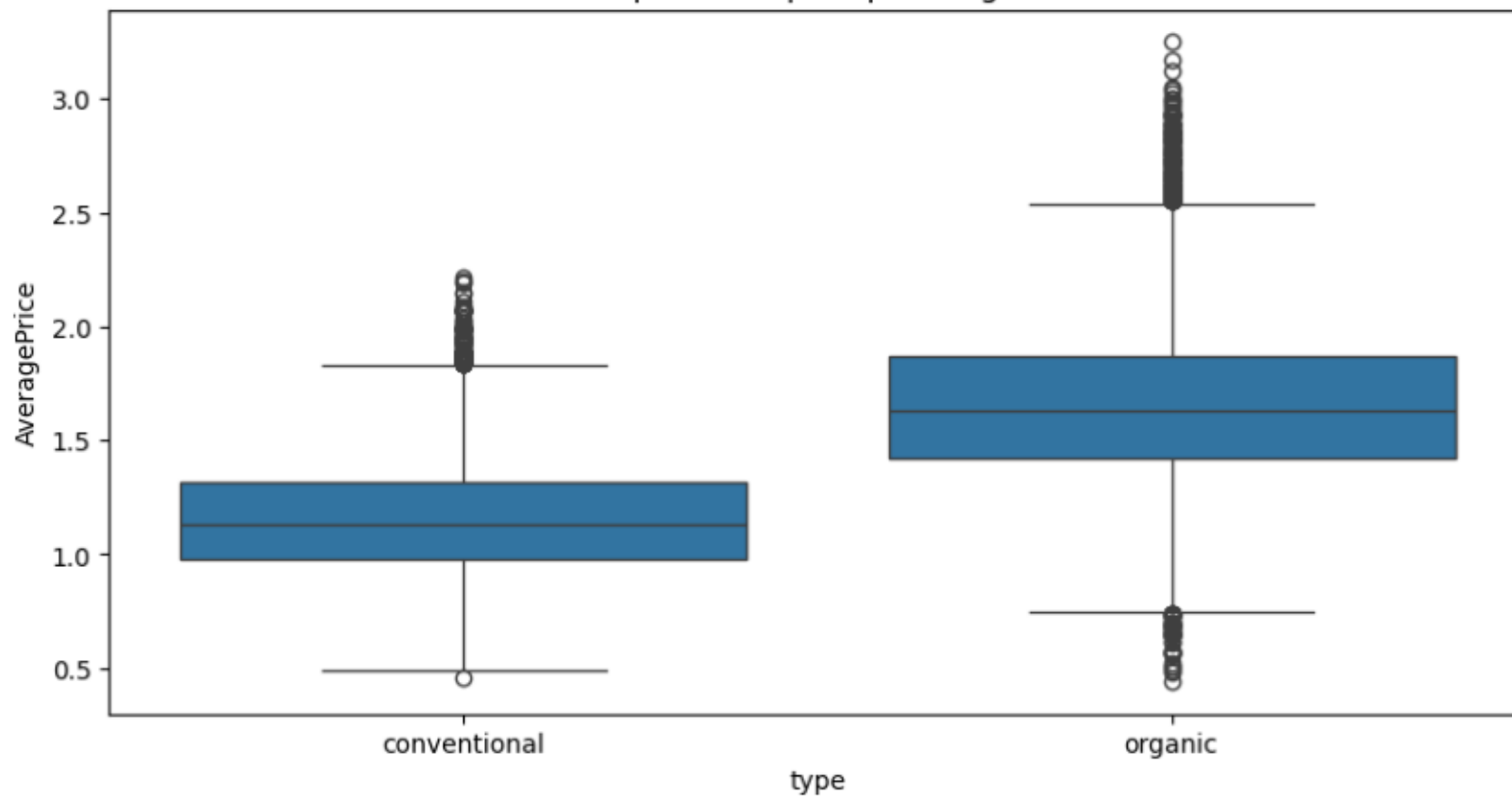


Aguacate - pred_vs_real.png
Predicción vs Realidad - Precio de Aguacate



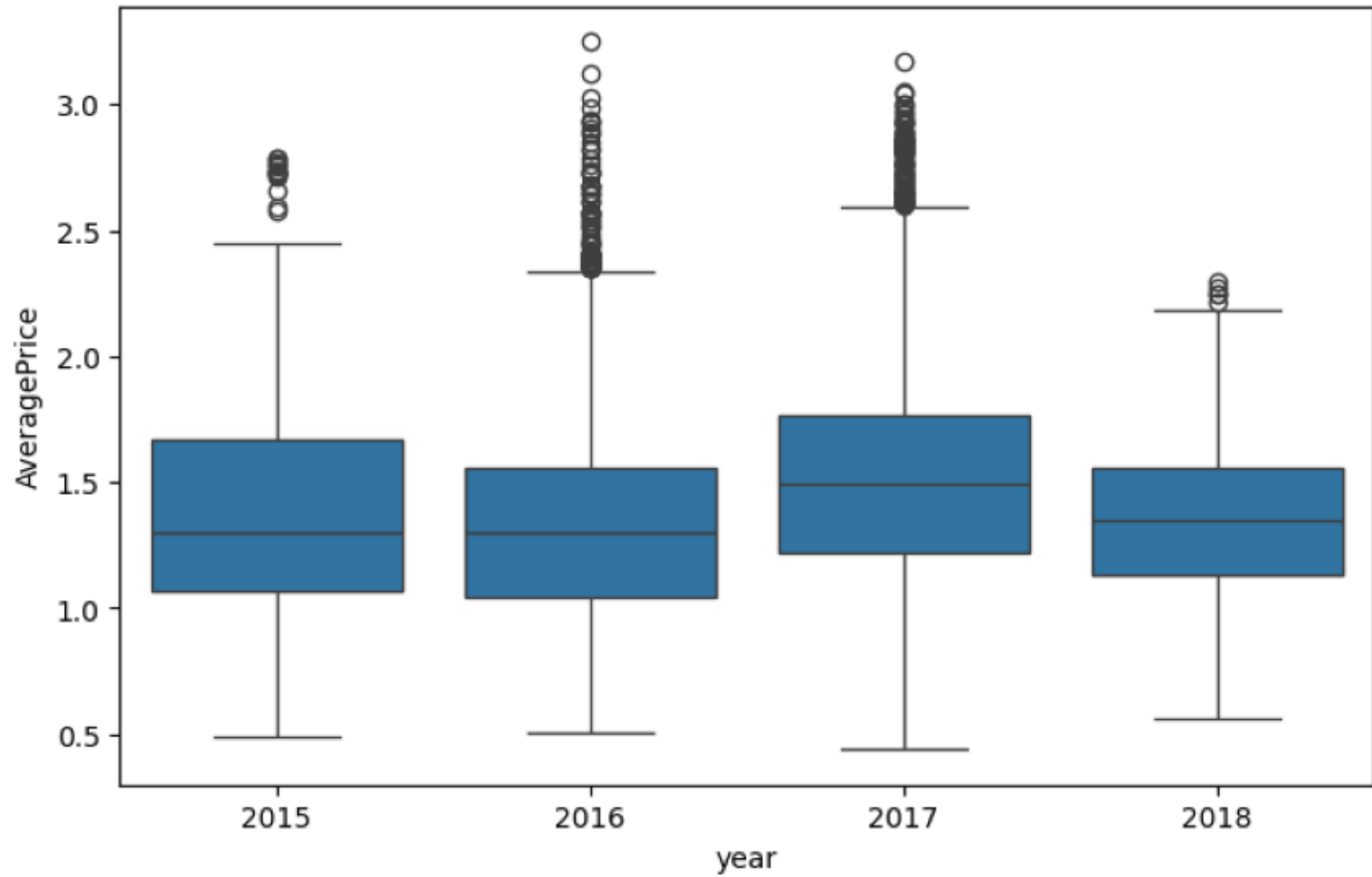
Aguacate - price_by_type.png

Precio promedio por tipo de aguacate



Aguacate - price_by_year.png

Precio promedio por año



Modelo: Bitcoin

Se predice la dirección del precio de Bitcoin (sube o baja al día siguiente) usando indicadores como precio de apertura, cierre, volumen, medias móviles y volatilidad. Modelo: Random Forest Classifier.

Métricas del modelo Bitcoin

Unnamed: 0	precision	recall	f1-score	support
0	0.5806451612903226	0.3857142857142857	0.463519313304721	140.0
1	0.4941176470588235	0.6829268292682927	0.5733788395904437	123.0
accuracy	0.5247148288973384	0.5247148288973384	0.5247148288973384	0.5247148288973384
macro avg	0.537381404174573	0.5343205574912893	0.5184490764475823	263.0
weighted avg	0.5401779207942222	0.5247148288973384	0.5148984833927206	263.0

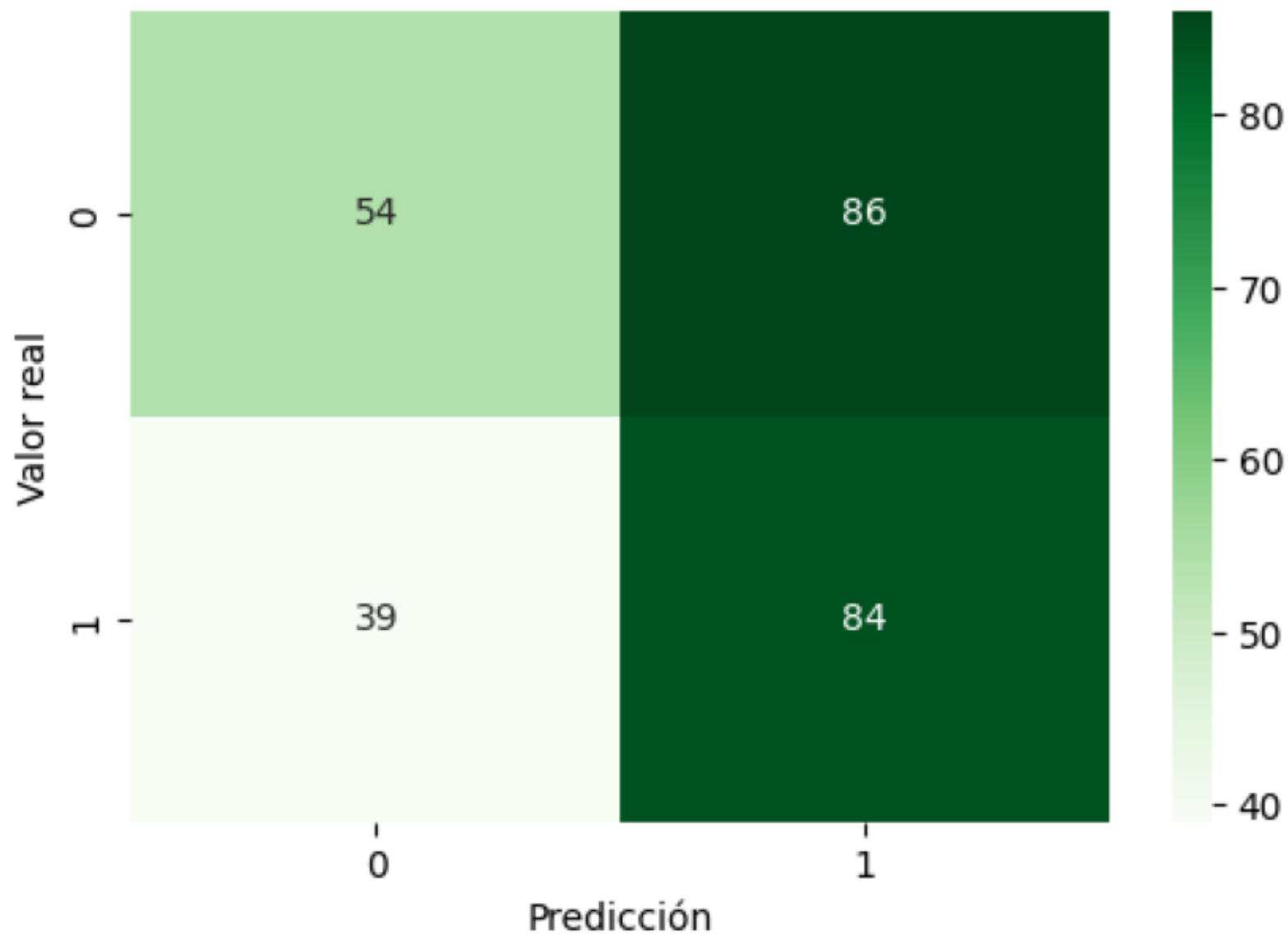
Bitcoin - 1_evolucion_precio.png

Evolución del precio de cierre de Bitcoin

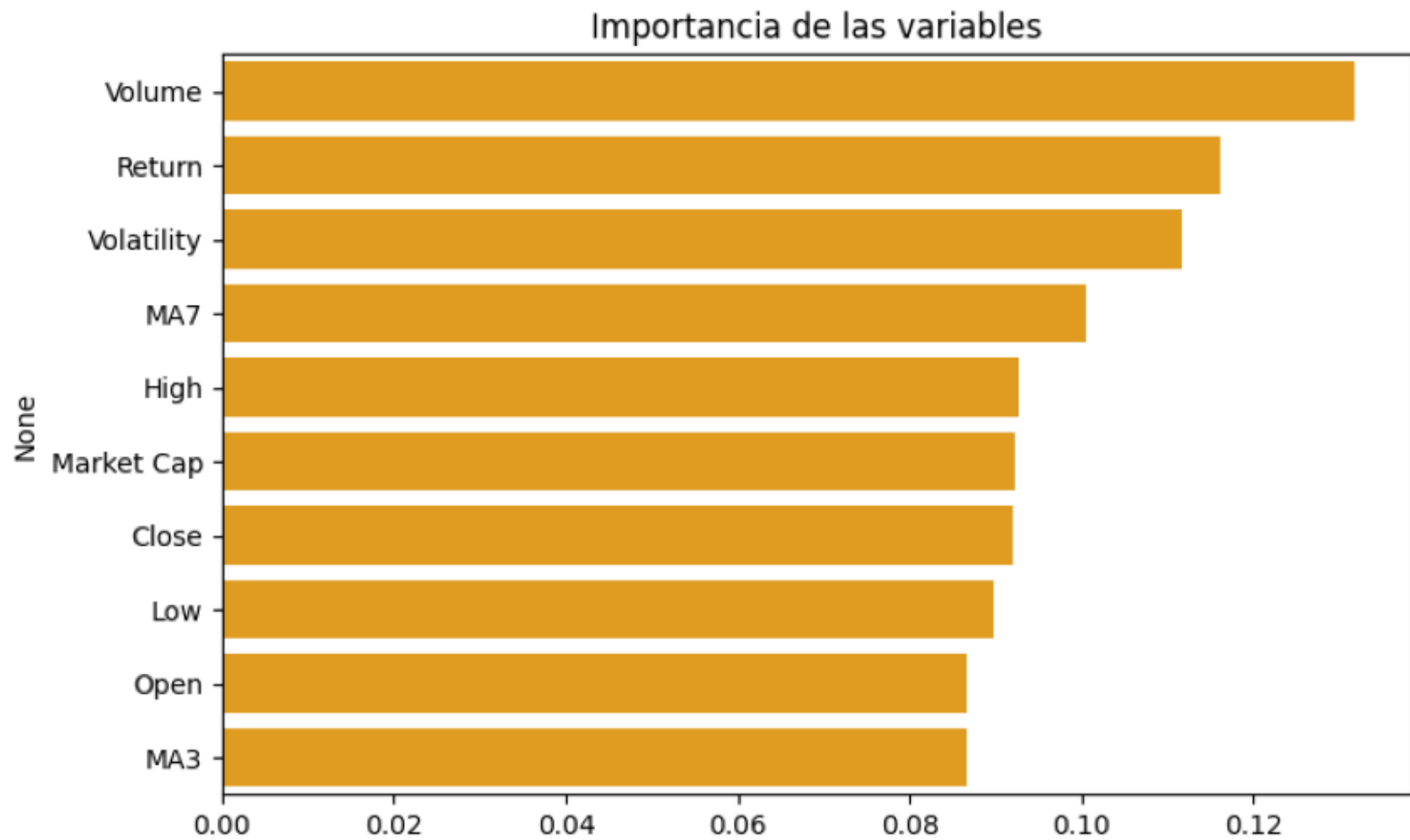


Bitcoin - 2_matriz_confusion.png

Matriz de confusión



Bitcoin - 3_importancia_variables.png



Modelo: BodyFat

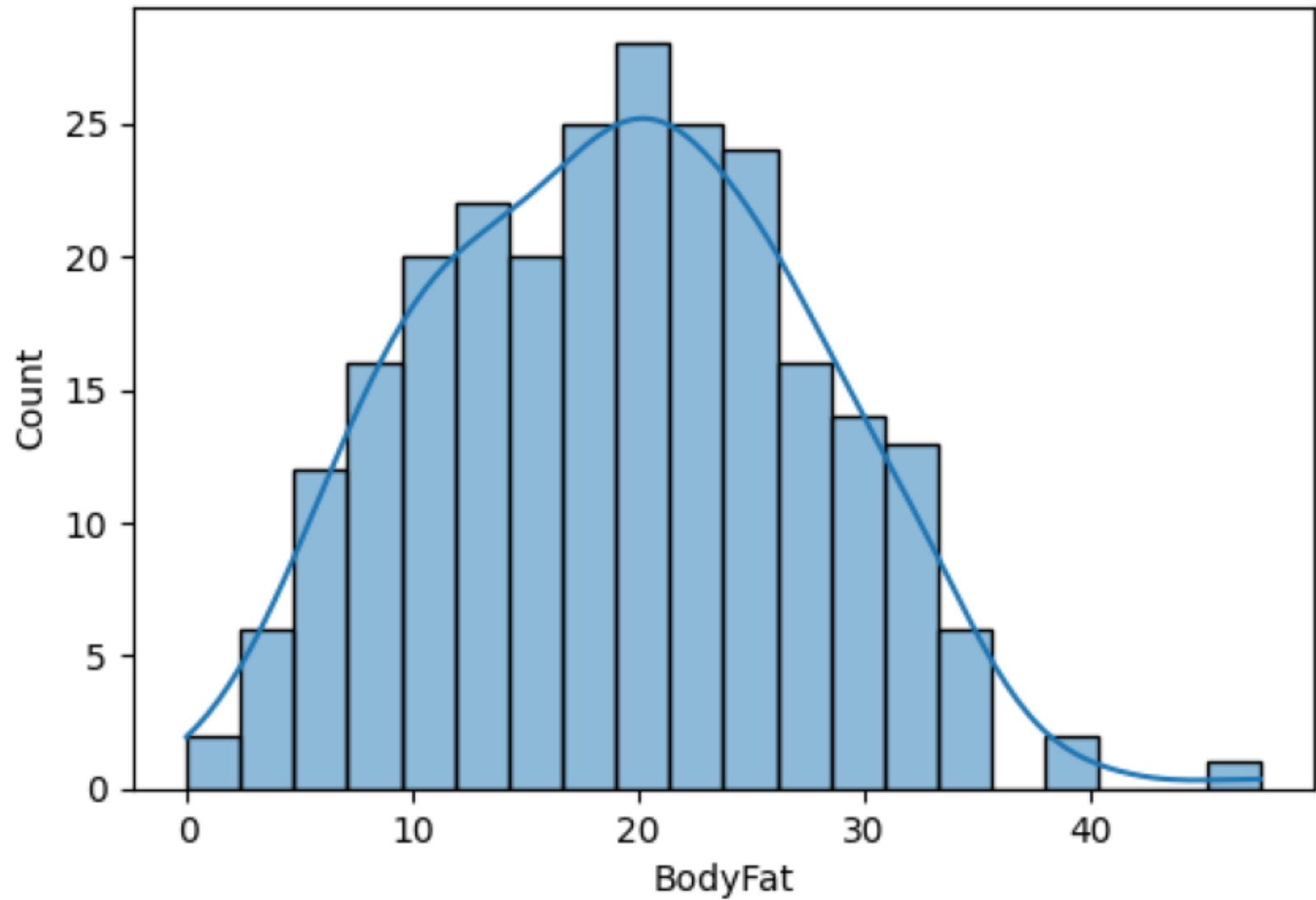
Se predice el porcentaje de grasa corporal (BodyFat) usando medidas antropométricas del cuerpo (edad, peso, altura, circunferencias, etc.), excluyendo la densidad. Modelo: Random Forest Regressor.

Métricas del modelo BodyFat

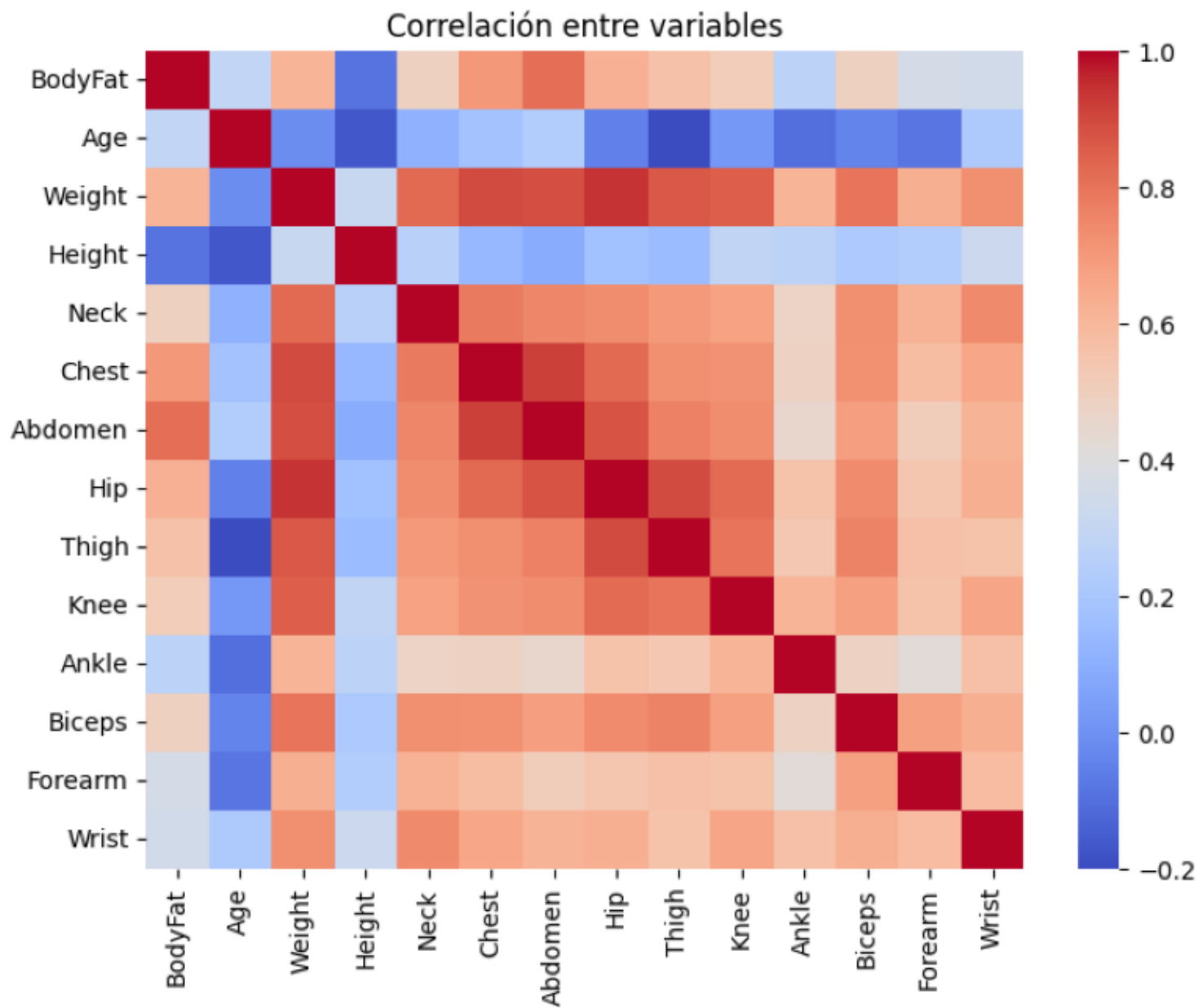
Métrica	Valor
MAE	3.9586470588235314
MSE	24.774759578431404
R ²	0.6187835600104352

BodyFat - bodyfat_distribution.png

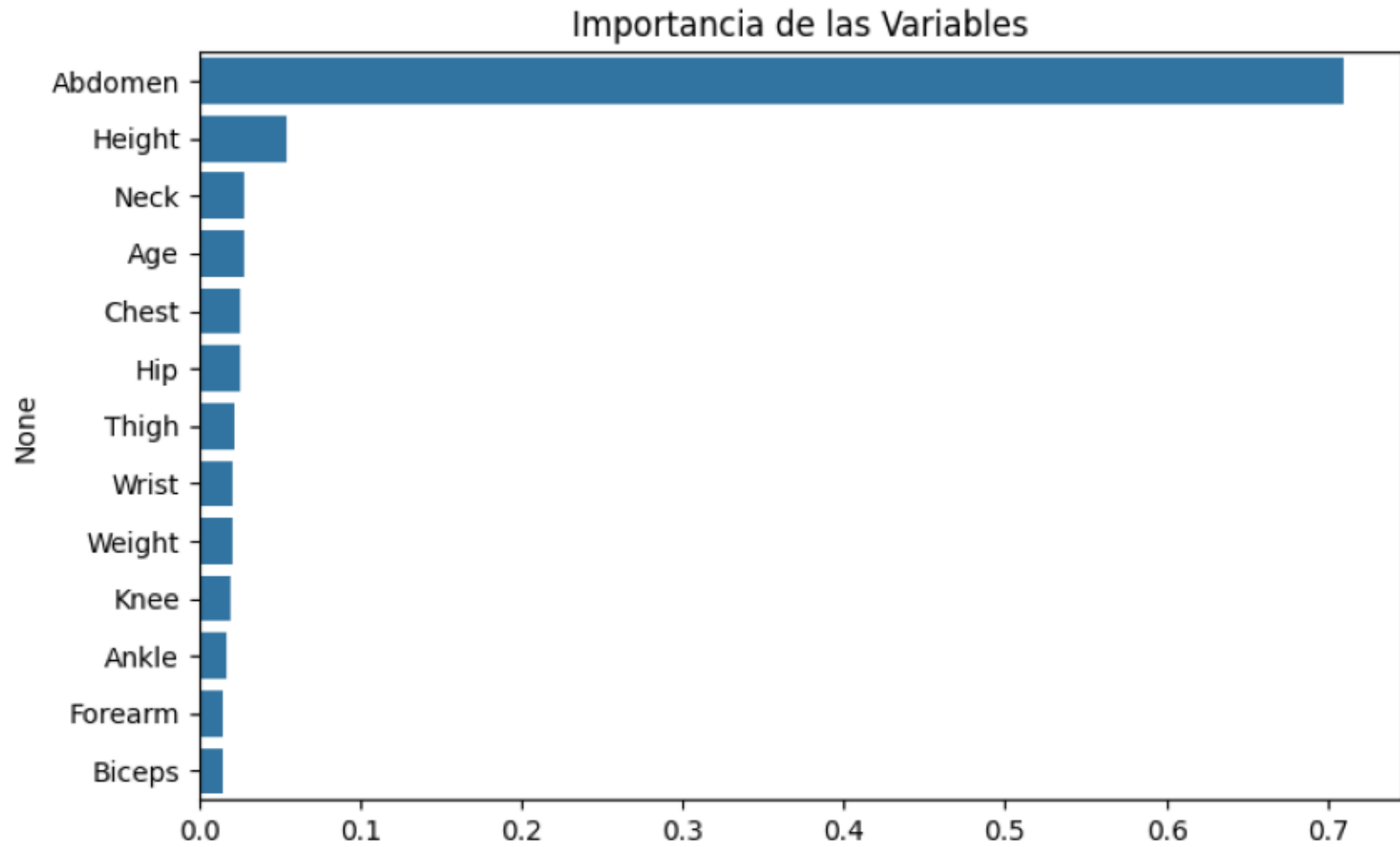
Distribución del BodyFat



BodyFat - correlation_matrix.png

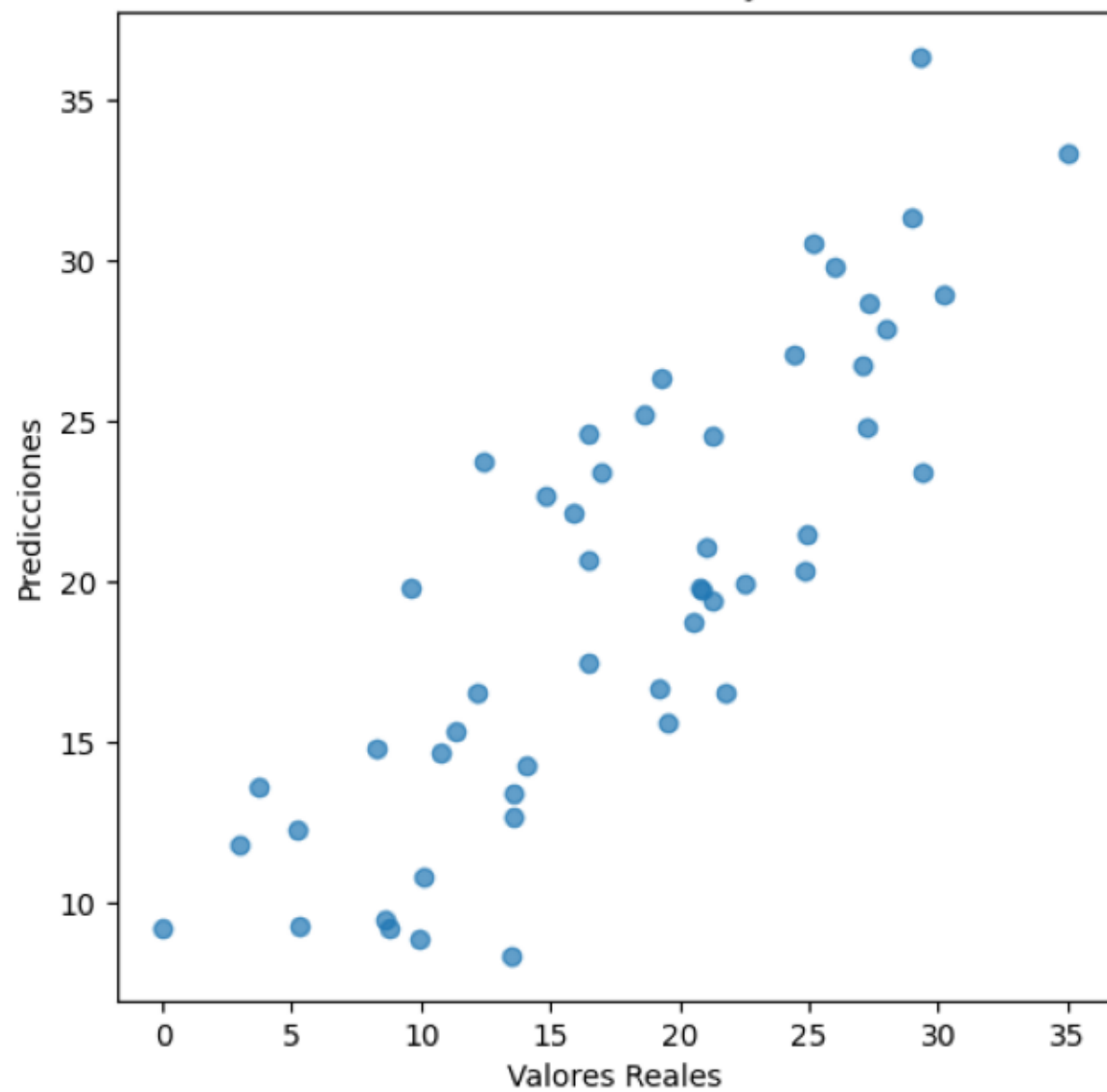


BodyFat - feature_importance.png



BodyFat - pred_vs_real.png

Predicción del BodyFat



Modelo: Telecom Churn

Se predice si un cliente de telecomunicaciones abandonará el servicio (Churn) según características demográficas, de contrato y uso de servicios. Modelo: Random Forest Classifier.

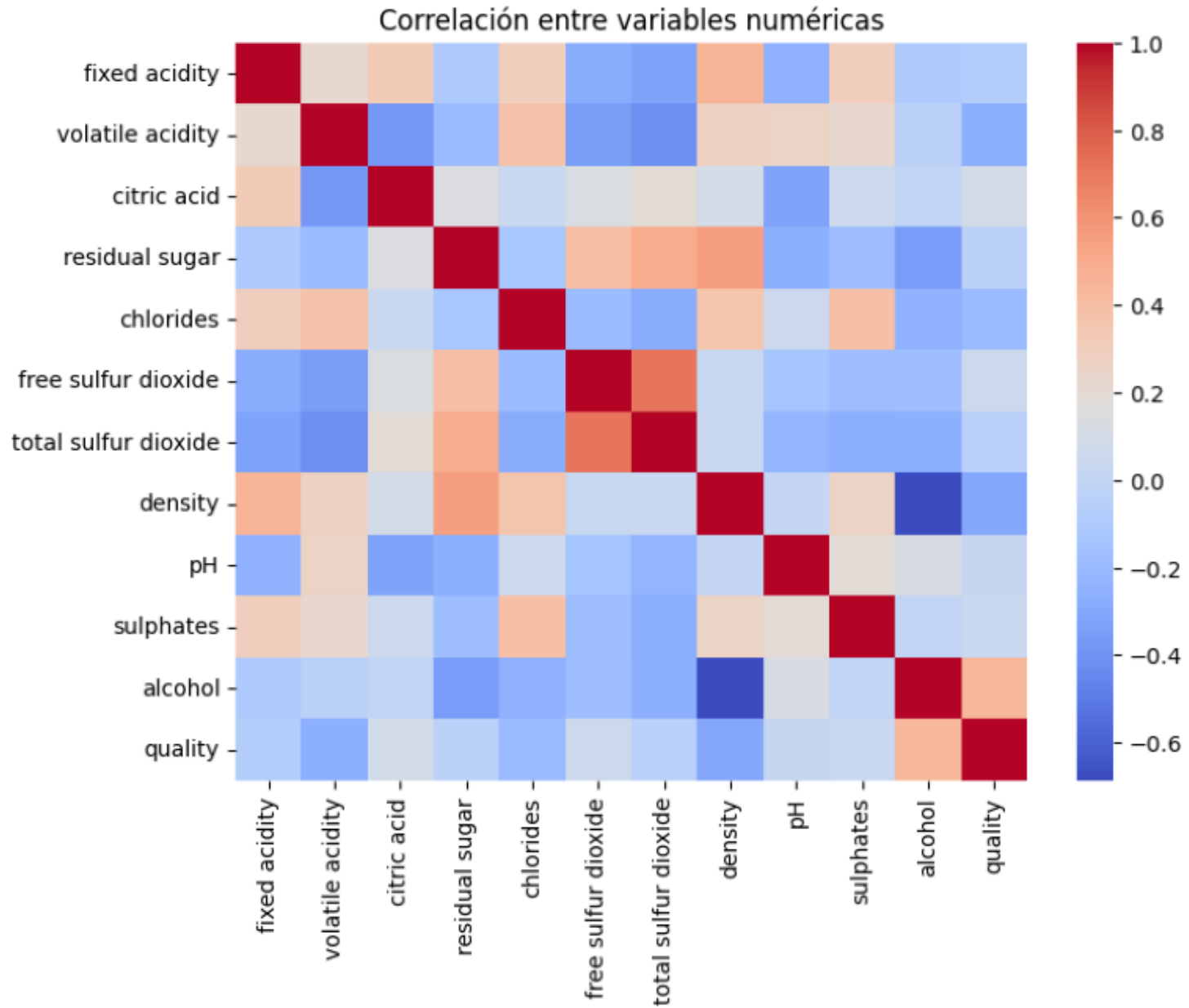
Modelo: Vino

Se predice la calidad del vino según sus propiedades químicas (acidez, azúcar, pH, alcohol, etc.) y tipo de vino. Modelo: Random Forest Regressor.

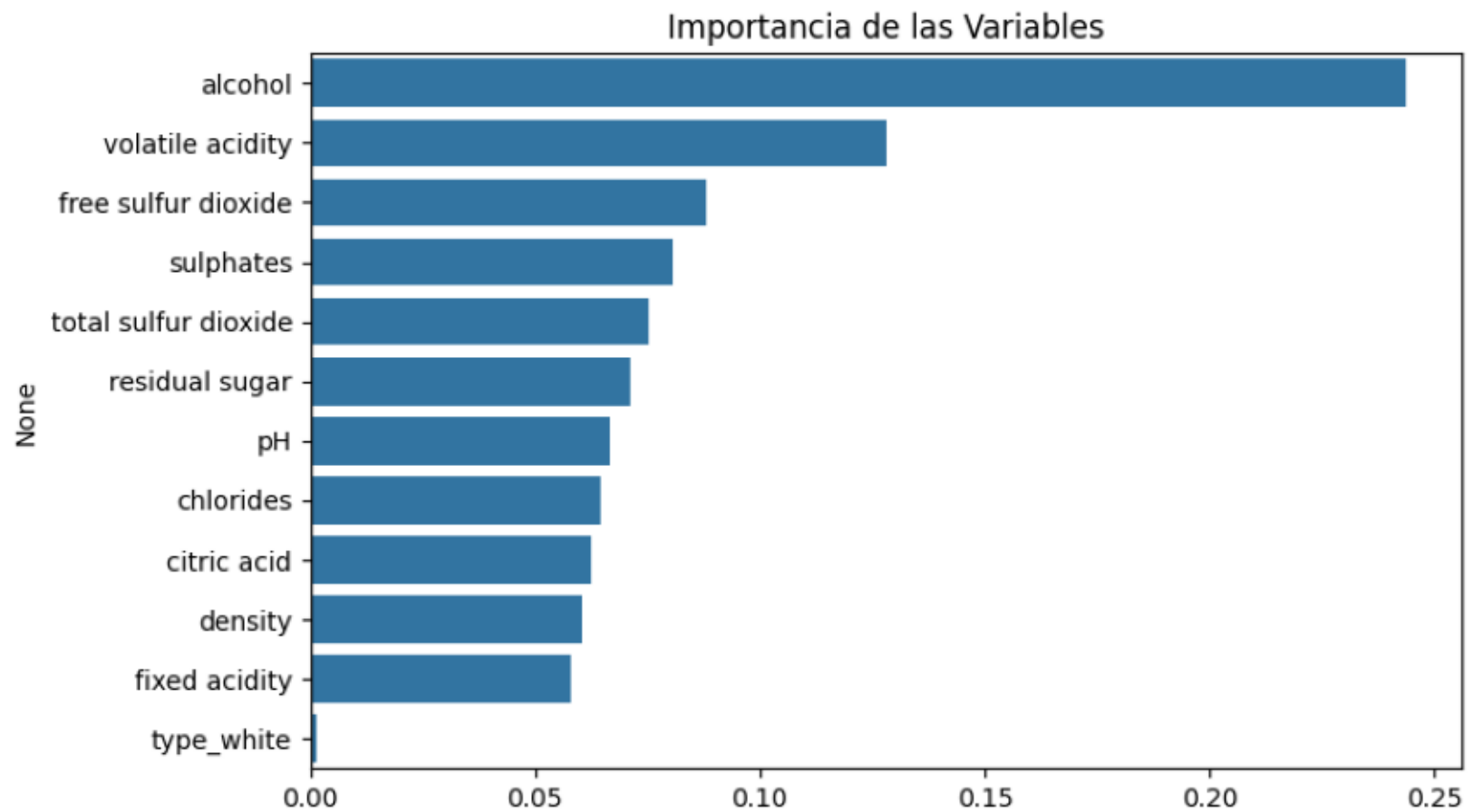
Métricas del modelo VINO

Métrica	Valor
MAE	0.4028692307692307
MSE	0.3142512692307692
R ²	0.5603643965162568

Vino - correlation_matrix.png

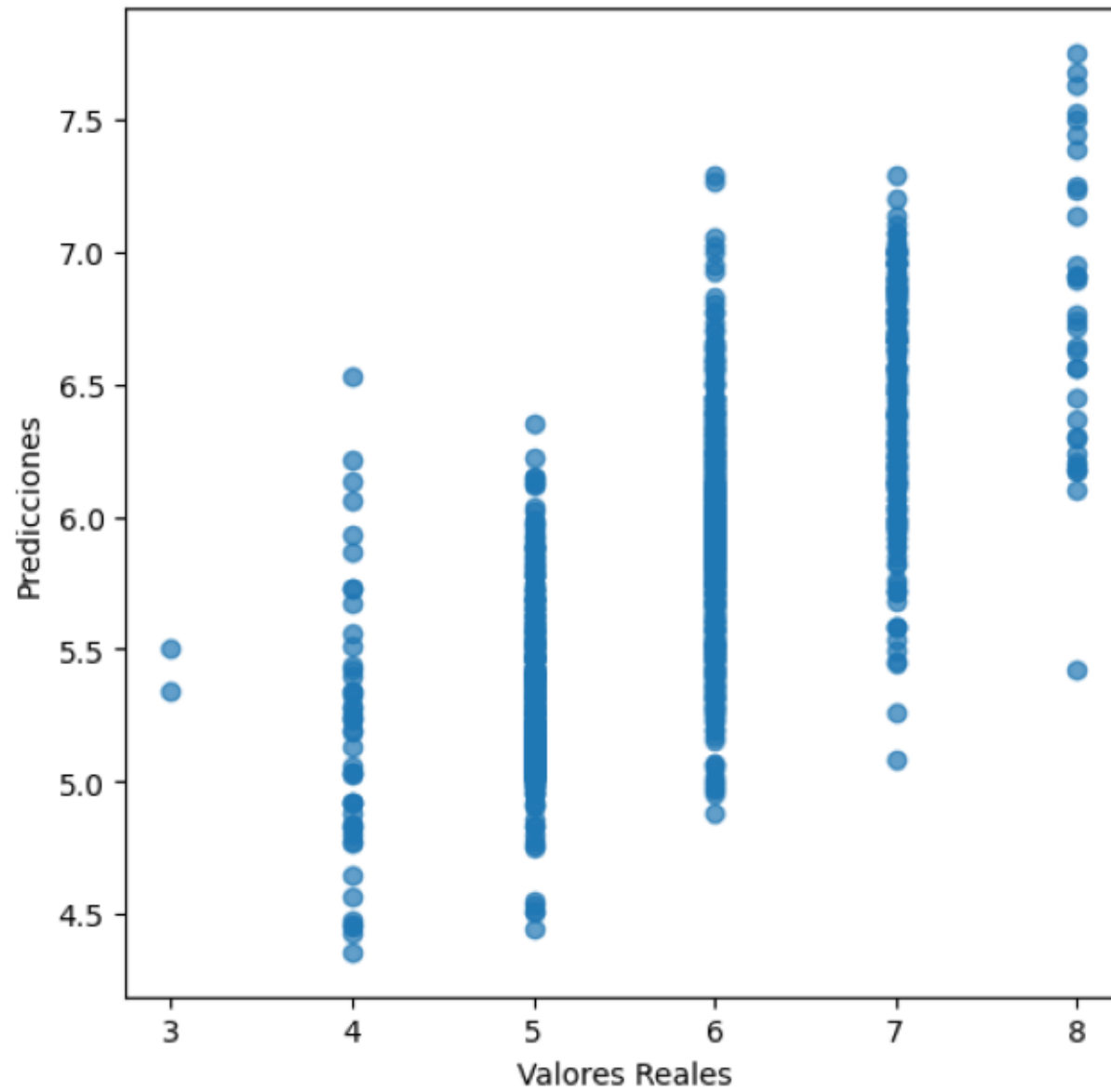


Vino - feature_importance.png



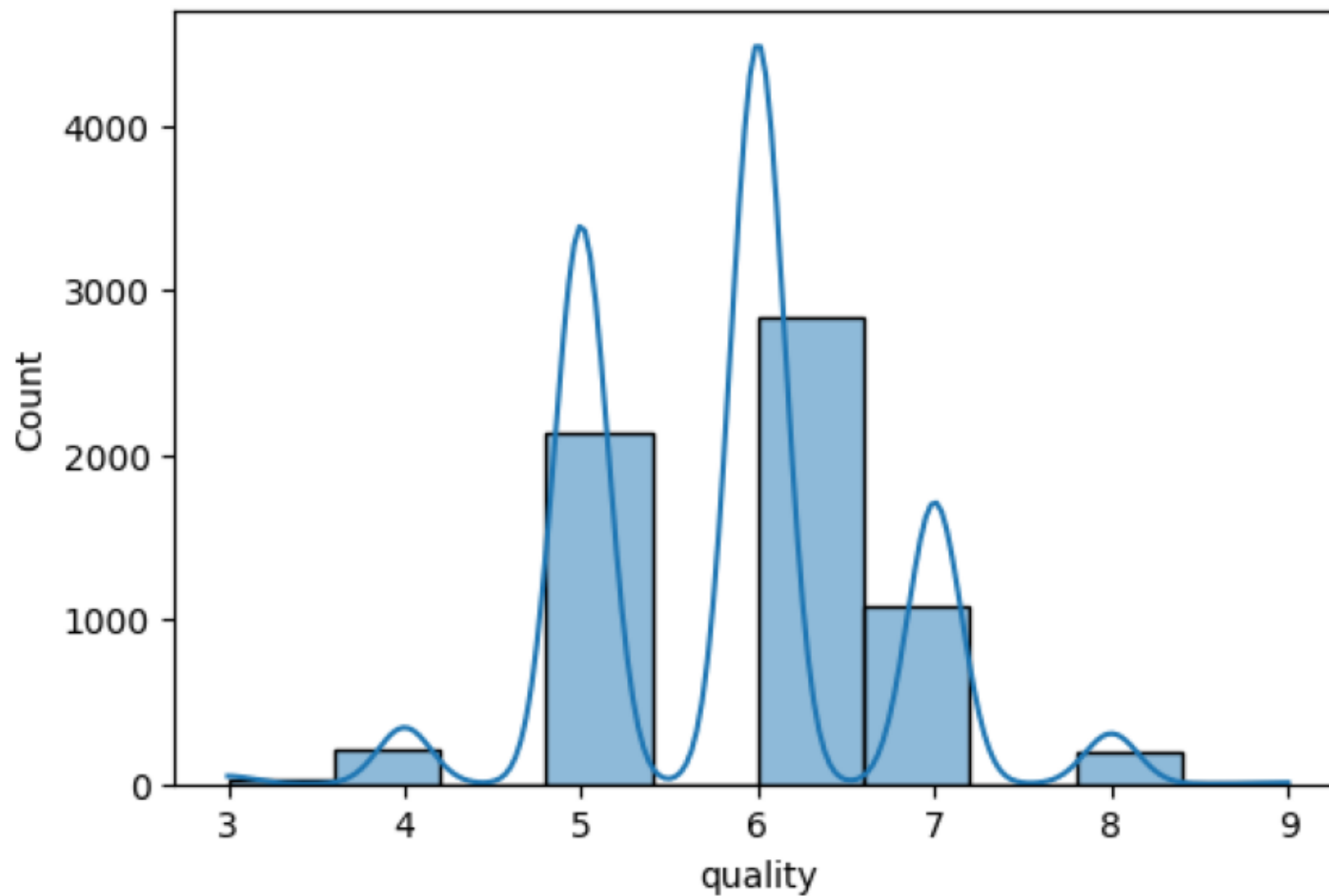
Vino - pred_vs_real.png

Predicción de calidad del vino



Vino - quality_distribution.png

Distribución de la calidad del vino



Código Python - Aguacate (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
data = pd.read_csv("data/aguacate.csv")

# Crear carpeta para figuras
os.makedirs("figures_aguacate", exist_ok=True)

# Visualización inicial
plt.figure(figsize=(8,5))
sns.histplot(data["AveragePrice"], bins=30, kde=True, color='green')
plt.title("Distribución del Precio Promedio del Aguacate")
plt.savefig("figures_aguacate/averageprice_distribution.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
sns.boxplot(x="year", y="AveragePrice", data=data)
plt.title("Precio promedio por año")
plt.savefig("figures_aguacate/price_by_year.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(10,5))
sns.boxplot(x="type", y="AveragePrice", data=data)
plt.title("Precio promedio por tipo de aguacate")
plt.savefig("figures_aguacate/price_by_type.png", bbox_inches="tight")
plt.close()

# Preparación de datos
data = data.drop(columns=["id", "Date"])

label_encoders = {}
for col in ["type", "region"]:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le

X = data.drop(columns=["AveragePrice"])
y = data["AveragePrice"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Código Python - Aguacate (líneas 51-86)

```
# Modelo de IA
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# Evaluación
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

results = pd.DataFrame({"Métrica": ["MAE", "RMSE", "R²"], "Valor": [mae, rmse, r2]})
results.to_csv("figures_aguacate/model_metrics.csv", index=False)
print("Evaluación del modelo:")
print(results)

# Visualización de resultados
plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Valores reales")
plt.ylabel("Predicciones")
plt.title("Predicción vs Realidad - Precio de Aguacate")
plt.savefig("figures_aguacate/pred_vs_real.png", bbox_inches="tight")
plt.close()

# Importancia de variables
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
plt.figure(figsize=(8,5))
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las variables en el modelo")
plt.savefig("figures_aguacate/feature_importance.png", bbox_inches="tight")
plt.close()

# Exportar modelo
joblib.dump(model, "modelo_precio_aguacate.pkl")
print("\nModelo exportado exitosamente a 'modelo_precio_aguacate.pkl'")
```

Código Python - Bitcoin (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import os
import joblib

# Crear carpeta para figuras
os.makedirs("figures_bitcoin", exist_ok=True)

# Cargar datos
data = pd.read_csv("data/bitcoin.csv")

# Limpiar columnas numéricas con comas y símbolos
for col in ["Volume", "Market Cap"]:
    data[col] = data[col].astype(str).replace({" ": ""}, regex=True)
    data[col] = data[col].replace(r"^[0-9\.\-]", "", regex=True)
    data[col] = pd.to_numeric(data[col], errors='coerce')

# Convertir fecha y ordenar
data["Date"] = pd.to_datetime(data["Date"])
data = data.sort_values("Date")

# Crear features
data["Return"] = data["Close"].pct_change()
data["MA3"] = data["Close"].rolling(3).mean()
data["MA7"] = data["Close"].rolling(7).mean()
data["Volatility"] = data["Close"].rolling(7).std()
data["Next_Close"] = data["Close"].shift(-1)
data["Direction"] = (data["Next_Close"] > data["Close"]).astype(int)
data = data.dropna()

# Variables predictoras y etiqueta
X = data[["Open", "High", "Low", "Close", "Volume", "Market Cap", "Return", "MA3", "MA7", "Volatility"]]
y = data["Direction"]

# División de datos
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=777, shuffle=True)

# Entrenamiento del modelo
model = RandomForestClassifier(n_estimators=200, random_state=777)
model.fit(X_train, y_train)

# Evaluación
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

Código Python - Bitcoin (líneas 51-83)

```
report_dict = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report_dict).transpose()
report_df.to_csv("figures_bitcoin/classification_report.csv")

print(f"Accuracy del modelo: {acc:.4f}")
print("\nReporte de clasificación:")
print(report_df)

# Visualizaciones
plt.figure(figsize=(10,5))
sns.lineplot(x="Date", y="Close", data=data)
plt.title("Evolución del precio de cierre de Bitcoin")
plt.savefig("figures_bitcoin/1_evolucion_precio.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens")
plt.title("Matriz de confusión")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig("figures_bitcoin/2_matriz_confusion.png", bbox_inches="tight")
plt.close()

importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
plt.figure(figsize=(8,5))
sns.barplot(x=importances.values, y=importances.index, color='orange')
plt.title("Importancia de las variables")
plt.savefig("figures_bitcoin/3_importancia_variables.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado
joblib.dump(model, "modelo_direccion_bitcoin.pkl")
print("\nModelo exportado a 'modelo_direccion_bitcoin.pkl' y figuras guardadas en /figures_bitcoin")
```

Código Python - BodyFat (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
df = pd.read_csv("data/bodyfat.csv")

# Crear carpeta para figuras
os.makedirs("figures_bodyfat", exist_ok=True)

# Visualización inicial
plt.figure(figsize=(8,6))
sns.heatmap(df.drop("Density", axis=1).corr(), annot=False, cmap="coolwarm")
plt.title("Correlación entre variables")
plt.savefig("figures_bodyfat/correlation_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.histplot(df["BodyFat"], bins=20, kde=True)
plt.title("Distribución del BodyFat")
plt.savefig("figures_bodyfat/bodyfat_distribution.png", bbox_inches="tight")
plt.close()

# Preparación de datos
X = df.drop(["BodyFat", "Density"], axis=1)
y = df["BodyFat"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=777)

# Entrenamiento del modelo
model = RandomForestRegressor(n_estimators=200, random_state=777)
model.fit(X_train, y_train)

# Evaluación del modelo
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

results = pd.DataFrame({
    "Métrica": ["MAE", "MSE", "R²"],
    "Valor": [mae, mse, r2]
```

Código Python - BodyFat (líneas 51-74)

```
)  
results.to_csv("figures_bodyfat/model_metrics.csv", index=False)  
print("Resultados del modelo:")  
print(results)  
  
# Visualización de resultados  
plt.figure(figsize=(6,6))  
plt.scatter(y_test, y_pred, alpha=0.7)  
plt.xlabel("Valores Reales")  
plt.ylabel("Predicciones")  
plt.title("Predicción del BodyFat")  
plt.savefig("figures_bodyfat/pred_vs_real.png", bbox_inches="tight")  
plt.close()  
  
plt.figure(figsize=(8,5))  
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)  
sns.barplot(x=importances.values, y=importances.index)  
plt.title("Importancia de las Variables")  
plt.savefig("figures_bodyfat/feature_importance.png", bbox_inches="tight")  
plt.close()  
  
# Guardar modelo entrenado  
joblib.dump(model, "bodyfat_model.pkl")  
print("\nModelo exportado exitosamente a 'bodyfat_model.pkl'")
```


Código Python - Telecom Churn (líneas 1-50)

```
# Importar librerías

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import joblib
import os

# Cargar datos

df = pd.read_csv("data/telecomunicaciones.csv")

# Crear carpeta para figuras

os.makedirs("figures_telecomunicaciones", exist_ok=True)

# Preprocesamiento

df = df.drop("customerID", axis=1)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors='coerce')
df = df.dropna()

categorical_cols = df.select_dtypes(include="object").columns
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

X = df.drop("Churn", axis=1)
y = df["Churn"]

# División de datos

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Entrenamiento del modelo

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
```

Código Python - Telecom Churn (líneas 51-83)

```
# Evaluación del modelo

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv("figures_telecomunicaciones/classification_report.csv")

print("Accuracy del modelo:", acc)
print("\nReporte de clasificación:")
print(report_df)

plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de Confusión")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig("figures_telecomunicaciones/confusion_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las variables")
plt.savefig("figures_telecomunicaciones/feature_importance.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado

joblib.dump(model, "telecomunicaciones_model.pkl")
```

Código Python - Vino (líneas 1-50)

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os

# Cargar datos
df = pd.read_csv("data/vino.csv")

os.makedirs("figures_vino_quality", exist_ok=True)

plt.figure(figsize=(8,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=False, cmap="coolwarm")
plt.title("Correlación entre variables numéricas")
plt.savefig("figures_vino_quality/correlation_matrix.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(6,4))
sns.histplot(df["quality"], bins=10, kde=True)
plt.title("Distribución de la calidad del vino")
plt.savefig("figures_vino_quality/quality_distribution.png", bbox_inches="tight")
plt.close()

# Preparación de datos
X = df.drop("quality", axis=1)
X = pd.get_dummies(X, drop_first=True) # Codifica 'type' y otras categóricas
y = df["quality"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenamiento del modelo
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# Evaluación del modelo
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
results = pd.DataFrame({"Métrica":["MAE", "MSE", "R²"], "Valor":[mae, mse, r2]})
results.to_csv("figures_vino_quality/model_metrics.csv", index=False)
print("Resultados del modelo:")
print(results)
```

Código Python - Vino (líneas 51-69)

```
# Visualización de resultados
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.xlabel("Valores Reales")
plt.ylabel("Predicciones")
plt.title("Predicción de calidad del vino")
plt.savefig("figures_vino_quality/pred_vs_real.png", bbox_inches="tight")
plt.close()

plt.figure(figsize=(8,5))
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index)
plt.title("Importancia de las Variables")
plt.savefig("figures_vino_quality/feature_importance.png", bbox_inches="tight")
plt.close()

# Guardar modelo entrenado
joblib.dump(model, "vino_quality_model.pkl")
```

Modelo: Bike Price

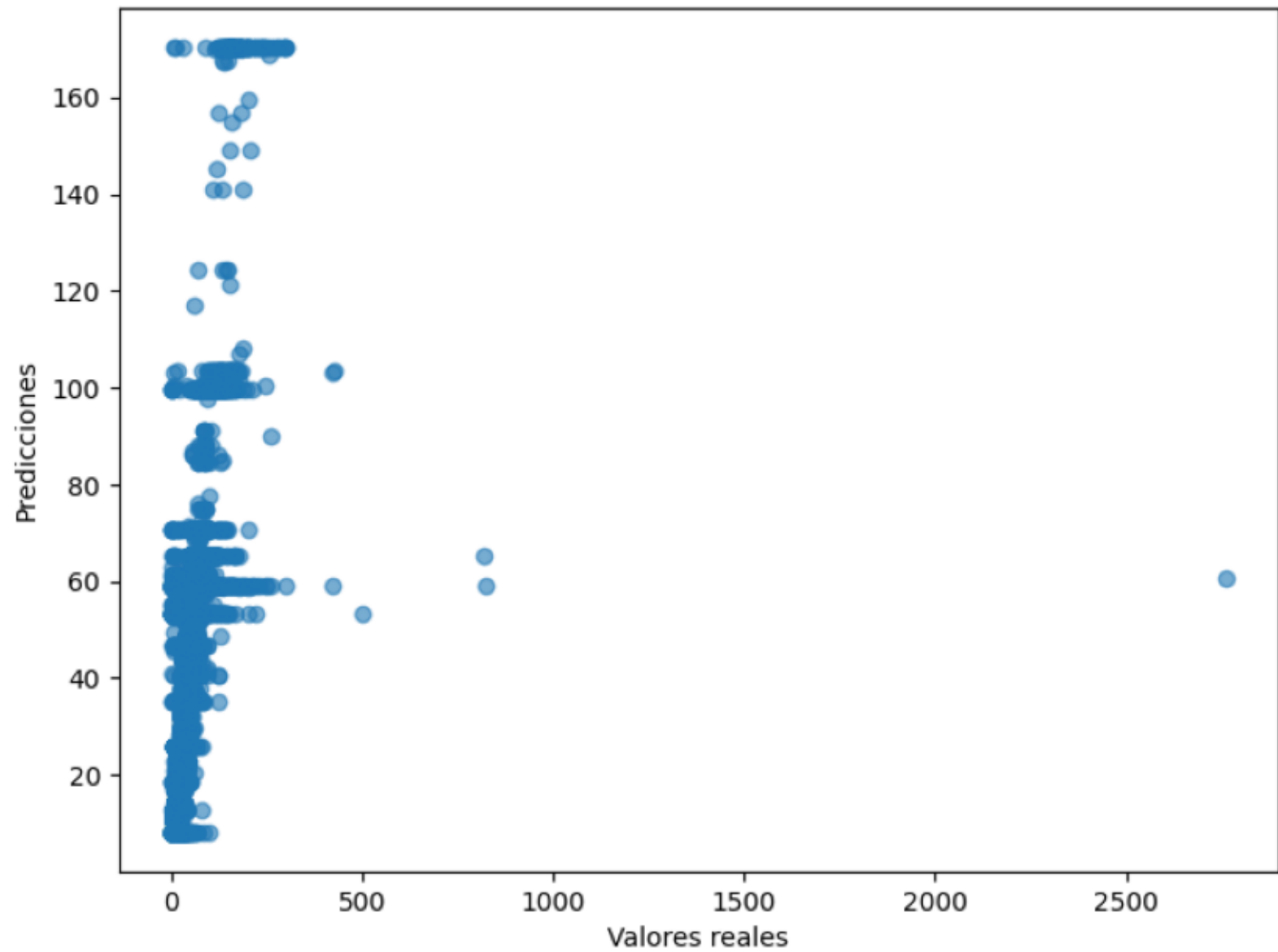
Predicción del precio total de un viaje en bicicleta (regresión). Variables: distancia, rate-code y otras características del viaje.

Métricas del modelo Bike Price

Métrica	Valor
R2	0.7766924452487566
RMSE	7.142996047443573

Bike Price - pred_vs_real_test.png

Predicción vs Real - Bike price (test)



Modelo: Car Price

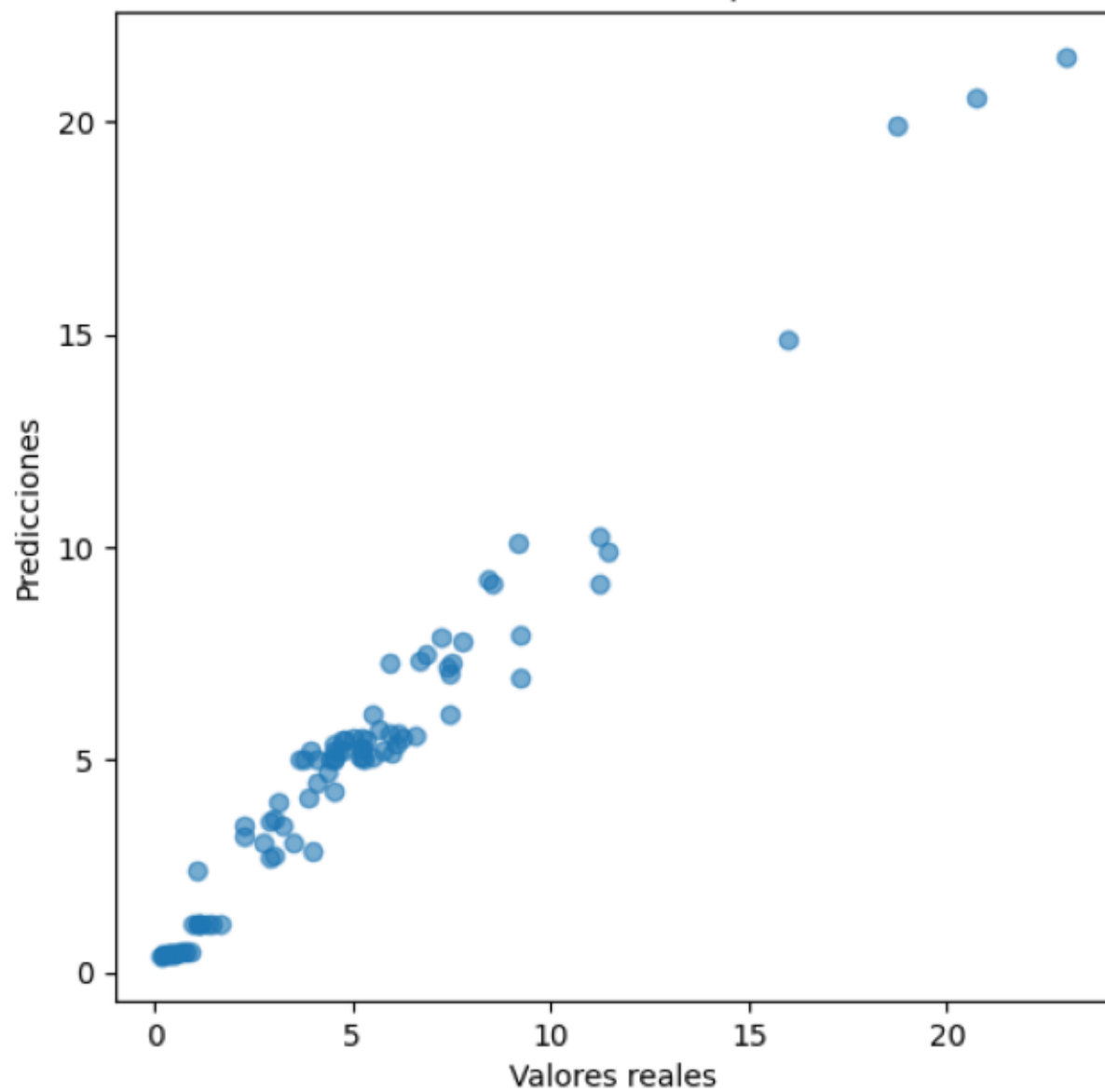
Predicción del precio de venta de un automóvil (regresión). Variables: características codificadas del vehículo (Car_Name_Code, Year, Present_Price, etc.).

Métricas del modelo Car Price

Métrica	Valor
R2	0.9693409270587292
RMSE	0.7408576828696707

Car Price - pred_vs_real_test.png

Predicción vs Real - Car price (test)



Modelo: Cirrhosis

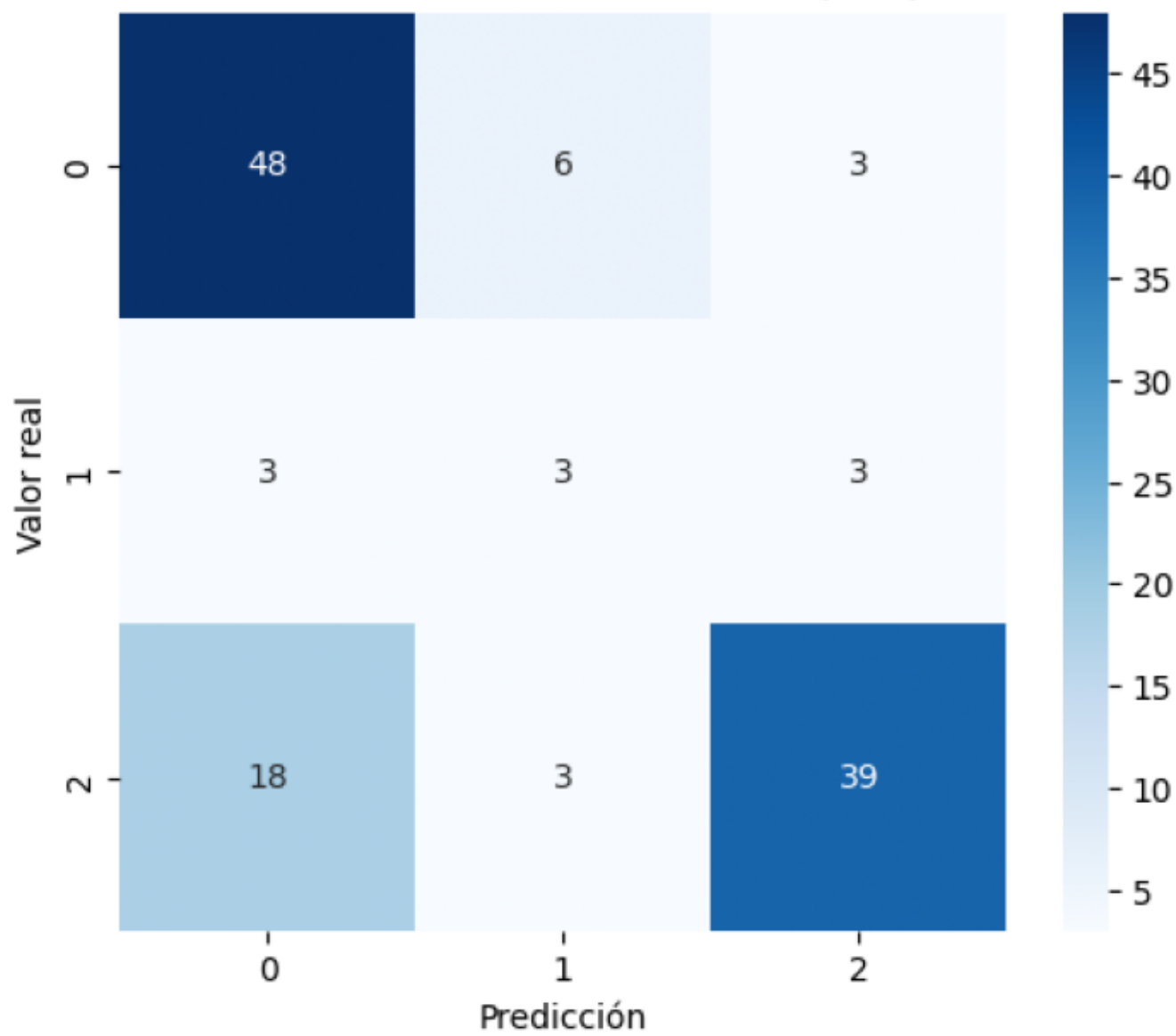
Clasificación del estado del paciente con cirrosis (multiclase). Variables clínicas y codificaciones (Drug, Sex, Ascites, etc.).

Métricas del modelo Cirrhosis

Unnamed: 0	precision	recall	f1-score	support
0	0.6956521739130435	0.8421052631578947	0.7619047619047619	57.0
1	0.25	0.3333333333333333	0.2857142857142857	9.0
2	0.8666666666666667	0.65	0.7428571428571429	60.0
accuracy	0.7142857142857143	0.7142857142857143	0.7142857142857143	0.7142857142857143
macro avg	0.6041062801932368	0.6084795321637427	0.5968253968253968	126.0
weighted avg	0.745255348516218	0.7142857142857143	0.7188208616780044	126.0

Cirrhosis - confusion_matrix_test.png

Matriz de confusión - Cirrhosis (test)



Modelo: Hepatitis

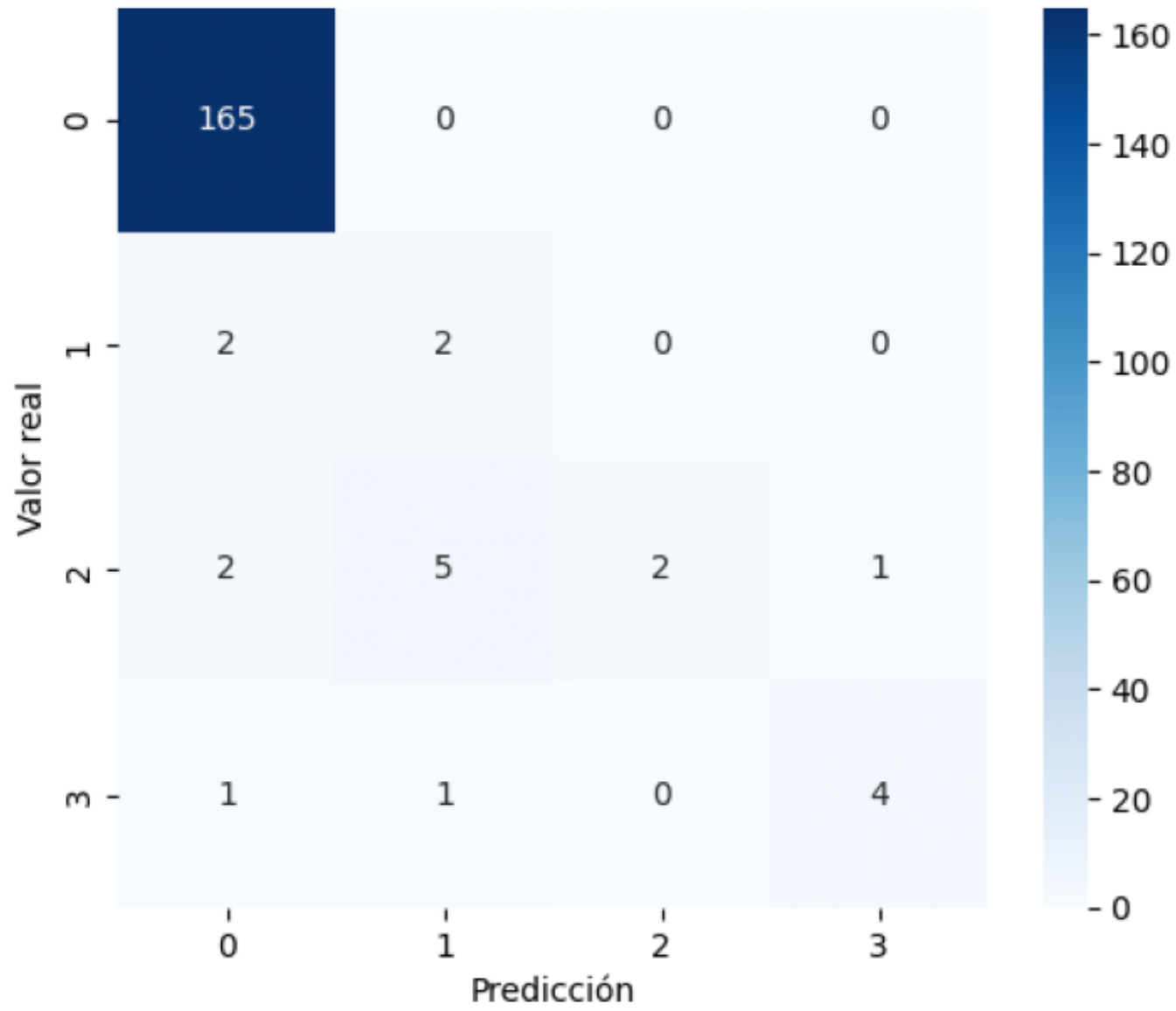
Clasificación de la categoría de Hepatitis C (multiclase) según parámetros bioquímicos y demográficos.

Métricas del modelo Hepatitis

Unnamed: 0	precision	recall	f1-score	support
0	0.9705882352941176	1.0	0.9850746268656716	165.0
2	0.25	0.5	0.3333333333333333	4.0
3	1.0	0.2	0.3333333333333333	10.0
4	0.8	0.6666666666666666	0.7272727272727273	6.0
accuracy	0.9351351351351352	0.9351351351351352	0.9351351351351352	0.9351351351351352
macro avg	0.7551470588235294	0.5916666666666667	0.5947535052012664	185.0
weighted avg	0.9510651828298888	0.9351351351351352	0.9273925214223724	185.0

Hepatitis - confusion_matrix_test.png

Matriz de confusión - Hepatitis (test)



Modelo: Stroke

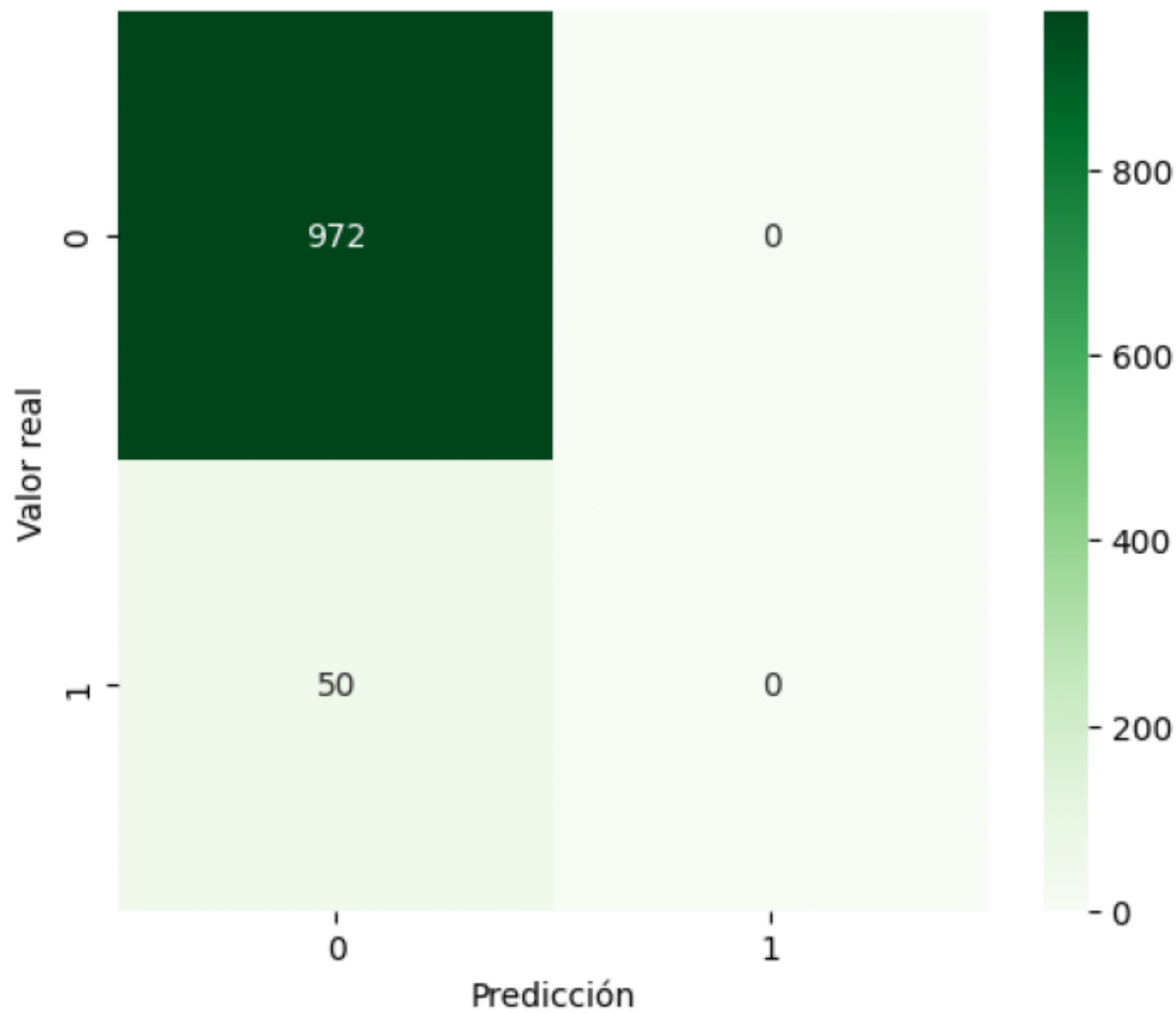
Predicción del riesgo de stroke (clasificación binaria) según edad, hipertensión, nivel de glucosa, BMI y otros.

Métricas del modelo Stroke

Unnamed: 0	precision	recall	f1-score	support
0	0.9510763209393346	1.0	0.9749247743229688	972.0
1	0.0	0.0	0.0	50.0
accuracy	0.9510763209393346	0.9510763209393346	0.9510763209393346	0.9510763209393346
macro avg	0.4755381604696673	0.5	0.4874623871614844	1022.0
weighted avg	0.9045461682515002	0.9510763209393346	0.9272278675557004	1022.0

Stroke - confusion_matrix_test.png

Matriz de confusión - Stroke (test)



Código Python - Bike Price (líneas 1-50)

```
import os
import math
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pandas import read_csv, DataFrame, Series
from numpy import ndarray
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.model_selection import train_test_split
from joblib import dump, load

def build_model(
    x_train: DataFrame | Series, y_train: DataFrame | Series | ndarray
) -> None:
    model = RandomForestRegressor(
        n_estimators=150, max_depth=4, random_state=67, min_samples_split=24
    )
    model.fit(x_train, y_train)
    # export model
    base_dir = os.path.dirname(__file__)
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)

    dump(model, os.path.join(base_dir, "bike Ride Price.model.pkl"))

    # Evaluación sobre los datos provistos (rápido sanity check)
    try:
        y_pred = model.predict(x_train)
        r2 = r2_score(y_train, y_pred)
        rmse = math.sqrt(mean_squared_error(y_train, y_pred))
        metrics_df = pd.DataFrame({"Métrica": ["R2", "RMSE"], "Valor": [r2, rmse]})
        metrics_df.to_csv(os.path.join(figures_dir, "model_metrics.csv"), index=False)

        # Gráficos básicos
        plt.figure(figsize=(8,6))
        plt.scatter(y_train, y_pred, alpha=0.6)
        plt.xlabel("Valores reales")
        plt.ylabel("Predicciones")
        plt.title("Predicción vs Real - Bike price")
        plt.savefig(os.path.join(figures_dir, "pred_vs_real.png"), bbox_inches="tight")
        plt.close()

        # Si las columnas son numéricas, mostrar matriz de correlación
        try:
            df = pd.concat([x_train.reset_index(drop=True), pd.Series(y_train).reset_index(drop=True)], axis=1)
            corr = df.corr()
            plt.figure(figsize=(8,6))
            sns.heatmap(corr, cmap="coolwarm", center=0)
```

Código Python - Bike Price (líneas 51-100)

```
plt.title("Matriz de correlación - Bike price")
plt.savefig(os.path.join(figures_dir, "correlation_matrix.png"), bbox_inches="tight")
plt.close()
except Exception:
    pass
except Exception:
    # no bloquear el entrenamiento si la exportación falla
    pass

def split_data() -> None:
    data = read_csv("./train.csv")

    train_data, test_data = train_test_split(
        data, test_size=.3, random_state=67
    )
    train_data.to_csv("train_data.csv")
    test_data.to_csv("test_data.csv")

def test_model() -> None:
    base_dir = os.path.dirname(__file__)
    model_path = os.path.join(base_dir, "bike_ride_price.model.pkl")
    model: RandomForestRegressor = load(model_path)
    test = read_csv(os.path.join(base_dir, "test_data.csv"))
    test = test.dropna()
    # x_test = test[["driver-tip", "mta-tax", "distance", "toll-amount", "rate-code"]]
    x_test = test[["distance", "rate-code"]]
    y_true = test["total-amount"]
    y_pred = model.predict(x_test)
    print("R² =", r2_score(y_true, y_pred))
    from sklearn.metrics import mean_squared_error
    rmse = math.sqrt(mean_squared_error(y_true, y_pred))
    print("RMSE =", rmse)

    # export figures for test evaluation
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)
    try:
        import pandas as pd
        metrics_df = _pd.DataFrame({"Métrica": ["R2", "RMSE"], "Valor": [r2_score(y_true, y_pred), rmse]})
        metrics_df.to_csv(os.path.join(figures_dir, "model_metrics_test.csv"), index=False)

        plt.figure(figsize=(8,6))
        plt.scatter(y_true, y_pred, alpha=0.6)
        plt.xlabel("Valores reales")
        plt.ylabel("Predicciones")
        plt.title("Predicción vs Real - Bike price (test)")
        plt.savefig(os.path.join(figures_dir, "pred_vs_real_test.png"), bbox_inches="tight")
        plt.close()
    except Exception:
        pass
```

Código Python - Bike Price (líneas 101-116)

```
# NOTE: this model is questionable at best, but it'll do, the other models are
# literal crap
# R² = 0.7938826430474483
# RMSE = 6.862557308745646
# driver-tip, mta-tax, distance, toll-amount, rate-code
if __name__ == "__main__":
    test_model()
    # split_data()
    exit(0)
    data = read_csv("./train_data.csv", parse_dates=False)
    data = data.dropna()
    # x_train = data[["driver-tip", "mta-tax", "distance", "toll-amount", "rate-code"]]
    x_train = data[["distance", "rate-code"]]
    y_train = data["total-amount"]
    build_model(x_train, y_train)
```

Código Python - Car Price (líneas 1-50)

```
import os
import math
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pandas import read_csv, DataFrame, Series
from numpy import ndarray
from joblib import dump, load
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
from warnings import filterwarnings

def build_model(
    x_train: DataFrame | Series, y_train: DataFrame | Series | ndarray
) -> None:
    model = RandomForestRegressor(n_estimators=100, max_depth=4)
    model.fit(x_train, y_train)
    base_dir = os.path.dirname(__file__)
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)

    dump(model, os.path.join(base_dir, "car_price.model.pkl"))

    # Evaluación rápida sobre los datos proporcionados
    try:
        y_pred = model.predict(x_train)
        r2 = r2_score(y_train, y_pred)
        rmse = math.sqrt(mean_squared_error(y_train, y_pred))
        metrics_df = pd.DataFrame({"Métrica": ["R2", "RMSE"], "Valor": [r2, rmse]})
        metrics_df.to_csv(os.path.join(figures_dir, "model_metrics.csv"), index=False)

        plt.figure(figsize=(6,6))
        plt.scatter(y_train, y_pred, alpha=0.6)
        plt.xlabel("Valores reales")
        plt.ylabel("Predicciones")
        plt.title("Predicción vs Real - Car price")
        plt.savefig(os.path.join(figures_dir, "pred_vs_real.png"), bbox_inches="tight")
        plt.close()

        # Importance (si aplica)
        try:
            importances = pd.Series(model.feature_importances_, index=x_train.columns).sort_values(ascending=False)
            plt.figure(figsize=(8,5))
            sns.barplot(x=importances.values, y=importances.index)
            plt.title("Importancia de las variables - Car price")
            plt.savefig(os.path.join(figures_dir, "feature_importance.png"), bbox_inches="tight")
            plt.close()
        except Exception:
```

Código Python - Car Price (líneas 51-100)

```
        pass
    except Exception:
        pass

def split_data() -> None:
    data = read_csv("./car_data.csv")
    data["Car_Name"] = data["Car_Name"].astype("category")
    data["Fuel_Type"] = data["Fuel_Type"].astype("category")
    data["Seller_Type"] = data["Seller_Type"].astype("category")
    data["Transmission"] = data["Transmission"].astype("category")

    data["Car_Name_Code"] = data["Car_Name"].cat.codes
    data["Fuel_Type_Code"] = data["Fuel_Type"].cat.codes
    data["Seller_Type_Code"] = data["Seller_Type"].cat.codes
    data["Transmission_Code"] = data["Transmission"].cat.codes

    # DataFrame({ "Car_Name": data["Car_Name"], "Car_Name_Code": data["Car_Name_Code"]}).to_csv("car_name_and_codes.csv")
    # DataFrame({ "Fuel_Type": data["Fuel_Type"], "Fuel_Type_Code": data["Fuel_Type_Code"]}).to_csv("fuel_type_and_codes.csv")
    # DataFrame({ "Seller_Type": data["Seller_Type"], "Seller_Type_Code": data["Seller_Type_Code"]}).to_csv("seller_type_and_codes.csv")
    # DataFrame({ "Transmission": data["Transmission"], "Transmission_Code": data["Transmission_Code"]}).to_csv("transmission_and_codes.csv")
    # exit(0)
    train_data: DataFrame
    test_data: DataFrame
    train_data, test_data = train_test_split(
        data, test_size=0.3, random_state=67
    ) # type:ignore
    train_data.to_csv("train_data.csv")
    test_data.to_csv("test_data.csv")

def test_model() -> None:
    base_dir = os.path.dirname(__file__)
    model_path = os.path.join(base_dir, "car_price.model.pkl")
    model: RandomForestRegressor = load(model_path)
    test = read_csv(os.path.join(base_dir, "test_data.csv"))
    x_test = test[
        [
            "Car_Name_Code",
            "Year",
            "Present_Price",
            "Kms_Driven",
            "Fuel_Type_Code",
            "Seller_Type_Code",
            "Transmission_Code",
            "Owner",
        ]
    ]
    y_true = test["Selling_Price"]
    y_pred = model.predict(x_test)
```


Código Python - Car Price (líneas 101-150)

```
x_test["predicted"] = y_pred
x_test["real_value"] = test["Selling_Price"]
x_test["difference"] = x_test["predicted"] - test["Selling_Price"]
print(y_true)
print("R² =", r2_score(y_true, y_pred))
rmse = math.sqrt(mean_squared_error(y_true, y_pred))
print("RMSE =", rmse)

# export figures for test evaluation
figures_dir = os.path.join(base_dir, "figures")
os.makedirs(figures_dir, exist_ok=True)
try:
    metrics_df = pd.DataFrame({"Métrica": ["R2", "RMSE"], "Valor": [r2_score(y_true, y_pred), rmse]})
    metrics_df.to_csv(os.path.join(figures_dir, "model_metrics_test.csv"), index=False)

    plt.figure(figsize=(6,6))
    plt.scatter(y_true, y_pred, alpha=0.6)
    plt.xlabel("Valores reales")
    plt.ylabel("Predicciones")
    plt.title("Predicción vs Real - Car price (test)")
    plt.savefig(os.path.join(figures_dir, "pred_vs_real_test.png"), bbox_inches="tight")
    plt.close()
except Exception:
    pass

# NOTE: got these results (based off of the R² metric this looks like a nice model)
# R² = 0.968924492355013
# RMSE = 0.7458721572349319

if __name__ == "__main__":
    filterwarnings("ignore")
    test_model()
    exit(0)
    train_data = read_csv("./train_data.csv")
    x_train = train_data[
        [
            "Car_Name_Code",
            "Year",
            "Present_Price",
            "Kms_Driven",
            "Fuel_Type_Code",
            "Seller_Type_Code",
            "Transmission_Code",
            "Owner",
        ]
    ]
    y_train = train_data["Selling_Price"]
    build_model(x_train, y_train)
```

Código Python - Cirrhosis (líneas 1-50)

```
from warnings import filterwarnings
import os
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pandas import DataFrame, Series, read_csv
from numpy import ndarray
from sklearn.metrics import classification_report, confusion_matrix
from joblib import dump, load
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

def build_model(
    x_train: DataFrame | Series, y_train: DataFrame | Series | ndarray
) -> None:
    model = DecisionTreeClassifier(
        max_depth=15, random_state=67, min_samples_split=24
    )
    model.fit(x_train, y_train)
    base_dir = os.path.dirname(__file__)
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)

    dump(model, os.path.join(base_dir, "cirrhosis.model.pkl"))

    # Exportar reporte y matrices básicas usando los mismos datos de entrenamiento (sanity check)
    try:
        y_pred = model.predict(x_train)
        report = classification_report(y_train, y_pred, output_dict=True)
        report_df = pd.DataFrame(report).transpose()
        report_df.to_csv(os.path.join(figures_dir, "classification_report.csv"))

        cm = confusion_matrix(y_train, y_pred)
        plt.figure(figsize=(6,5))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
        plt.title("Matriz de confusión - Cirrhosis")
        plt.xlabel("Predicción")
        plt.ylabel("Valor real")
        plt.savefig(os.path.join(figures_dir, "confusion_matrix.png"), bbox_inches="tight")
        plt.close()
    except Exception:
        pass

def split_data() -> None:
    data = read_csv("./cirrhosis.csv")
    data["Status"] = data["Status"].astype("category")
    data["Drug"] = data["Drug"].astype("category")
    data["Sex"] = data["Sex"].astype("category")
```

Código Python - Cirrhosis (líneas 51-100)

```
data["Ascites"] = data["Ascites"].astype("category")
data["Hepatomegaly"] = data["Hepatomegaly"].astype("category")
data["Spiders"] = data["Spiders"].astype("category")
data["Edema"] = data["Edema"].astype("category")

data["Status_Code"] = data["Status"].cat.codes
data["Drug_Code"] = data["Drug"].cat.codes
data["Sex_Code"] = data["Sex"].cat.codes
data["Ascites_Code"] = data["Ascites"].cat.codes
data["Hepatomegaly_Code"] = data["Hepatomegaly"].cat.codes
data["Spiders_Code"] = data["Spiders"].cat.codes
data["Edema_Code"] = data["Edema"].cat.codes

train_data, test_data = train_test_split(
    data, test_size=0.3, random_state=67
)
train_data.to_csv("train_data.csv")
test_data.to_csv("test_data.csv")

def test_model() -> None:
    base_dir = os.path.dirname(__file__)
    model: DecisionTreeClassifier = load(os.path.join(base_dir, "cirrhosis.model.pkl"))
    test = read_csv(os.path.join(base_dir, "test_data.csv"))
    x_test = test[
        [
            "N_Days",
            "Drug_Code",
            "Age",
            "Sex_Code",
            "Ascites_Code",
            "Hepatomegaly_Code",
            "Spiders_Code",
            "Edema_Code",
            "Bilirubin",
            "Cholesterol",
            "Albumin",
            "Copper",
            "Alk Phos",
            "SGOT",
            "Tryglicerides",
            "Platelets",
            "Prothrombin",
            "Stage",
        ]
    ]
    y_true = test["Status_Code"]
    y_pred = model.predict(x_test)
    from sklearn.metrics import accuracy_score, classification_report
    print("Accuracy =", accuracy_score(y_true, y_pred))
```

Código Python - Cirrhosis (líneas 101-150)

```
print("Reporte de clasificación:\n", classification_report(y_true, y_pred))

# export figures for test evaluation
figures_dir = os.path.join(base_dir, "figures")
os.makedirs(figures_dir, exist_ok=True)
try:
    report = classification_report(y_true, y_pred, output_dict=True)
    report_df = pd.DataFrame(report).transpose()
    report_df.to_csv(os.path.join(figures_dir, "classification_report_test.csv"))

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6,5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title("Matriz de confusión - Cirrhosis (test)")
    plt.xlabel("Predicción")
    plt.ylabel("Valor real")
    plt.savefig(os.path.join(figures_dir, "confusion_matrix_test.png"), bbox_inches="tight")
    plt.close()
except Exception:
    pass

if __name__ == "__main__":
    filterwarnings("ignore")
    test_model()
    exit(0)
    # split data()
    # exit(0)
    train_data = read_csv("./train_data.csv")
    train_data.dropna()
    x_train = train_data[
        [
            "N_Days",
            "Drug_Code",
            "Age",
            "Sex_Code",
            "Ascites_Code",
            "Hepatomegaly_Code",
            "Spiders_Code",
            "Edema_Code",
            "Bilirubin",
            "Cholesterol",
            "Albumin",
            "Copper",
            "Alk Phos",
            "SGOT",
            "Tryglicerides",
            "Platelets",
            "Prothrombin",
            "Stage",
        ]
    ]
```

Código Python - Cirrhosis (líneas 151-154)

```
    ]  
]  
y_train = train_data["Status_Code"]  
build_model(x_train, y_train)
```

Código Python - Hepatitis (líneas 1-50)

```
import os
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from pandas import read_csv, DataFrame, Series
from numpy import ndarray
from joblib import dump, load
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from warnings import filterwarnings

def build_model(
    x_train: DataFrame | Series, y_train: DataFrame | Series | ndarray
) -> None:
    model = RandomForestClassifier(n_estimators=100, max_depth=4)
    model.fit(x_train, y_train)
    base_dir = os.path.dirname(__file__)
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)

    dump(model, os.path.join(base_dir, "hepatitis.model.pkl"))

    # Exportar métricas básicas y figuras (sanity check)
    try:
        y_pred = model.predict(x_train)
        report = classification_report(y_train, y_pred, output_dict=True)
        report_df = pd.DataFrame(report).transpose()
        report_df.to_csv(os.path.join(figures_dir, "classification_report.csv"))

        cm = confusion_matrix(y_train, y_pred)
        plt.figure(figsize=(6,5))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
        plt.title("Matriz de confusión - Hepatitis")
        plt.xlabel("Predicción")
        plt.ylabel("Valor real")
        plt.savefig(os.path.join(figures_dir, "confusion_matrix.png"), bbox_inches="tight")
        plt.close()
    except Exception:
        pass

def split_data() -> None:
    data = read_csv("../HepatitisCdata.csv")
    data["Category"] = data["Category"].astype("category")
    data["Sex"] = data["Sex"].astype("category")

    data["Category_Code"] = data["Category"].cat.codes
    data["Sex_Code"] = data["Sex"].cat.codes
```

Código Python - Hepatitis (líneas 51-100)

```
DataFrame(
    {"Category": data["Category"], "Category_Code": data["Category_Code"]}
).to_csv("category_and_category_codes.csv")
DataFrame({"Sex": data["Sex"], "Sex_Code": data["Sex_Code"]}).to_csv(
    "sex_and_sex_codes.csv"
)

train_data: DataFrame
test_data: DataFrame
train_data, test_data = train_test_split(
    data, test_size=0.3, random_state=67
) # type:ignore
train_data.to_csv("train_data.csv")
test_data.to_csv("test_data.csv")

def test_model() -> None:
    base_dir = os.path.dirname(__file__)
    model: RandomForestClassifier = load(os.path.join(base_dir, "hepatitis.model.pkl"))
    test = read_csv(os.path.join(base_dir, "test_data.csv"))
    x_test = test[
        [
            "Age",
            "Sex Code",
            "ALB",
            "ALP",
            "ALT",
            "AST",
            "BIL",
            "CHE",
            "CHOL",
            "CREA",
            "GGT",
            "PROT",
        ]
    ]
    y_true = test["Category_Code"]
    y_pred = model.predict(x_test)
    from sklearn.metrics import accuracy_score, classification_report
    print("Accuracy =", accuracy_score(y_true, y_pred))
    print("Reporte de clasificación:\n", classification_report(y_true, y_pred))

    # export figures for test evaluation
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)
    try:
        report = classification_report(y_true, y_pred, output_dict=True)
        report_df = pd.DataFrame(report).transpose()
        report_df.to_csv(os.path.join(figures_dir, "classification_report_test.csv"))
```

Código Python - Hepatitis (líneas 101-125)

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de confusión - Hepatitis (test)")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig(os.path.join(figures_dir, "confusion_matrix_test.png"), bbox_inches="tight")
plt.close()
except Exception:
    pass

# NOTE: got these values, this also seems like a very solid model
# R² = 0.7130585217443179
# RMSE = 0.5301708036860207

if __name__ == "__main__":
    filterwarnings("ignore")
    test_model()
    exit(0)
    train_data = read_csv("./train_data.csv")
    x_train = train_data[["Age", "Sex_Code", "ALB", "ALP", "ALT", "AST", "BIL", "CHE", "CHOL", "CREA", "GGT", "PROT"]]
    y_train = train_data["Category_Code"]
    build_model(x_train, y_train)
```


Código Python - Stroke (líneas 1-50)

```
import os
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from joblib import load, dump
from pandas import DataFrame, Series, read_csv
from numpy.random import rand
from numpy import ndarray
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.tree import DecisionTreeClassifier

def build_model(
    x_train: DataFrame | Series, y_train: DataFrame | Series | ndarray
) -> None:
    model = RandomForestClassifier(n_estimators=200, max_depth=4)
    model.fit(x_train, y_train)
    base_dir = os.path.dirname(__file__)
    figures_dir = os.path.join(base_dir, "figures")
    os.makedirs(figures_dir, exist_ok=True)

    dump(model, os.path.join(base_dir, "stroke.model.pkl"))

    # Exportar métricas y figuras (sanity check)
    try:
        y_pred = model.predict(x_train)
        report = classification_report(y_train, y_pred, output_dict=True)
        report_df = pd.DataFrame(report).transpose()
        report_df.to_csv(os.path.join(figures_dir, "classification_report.csv"))

        cm = confusion_matrix(y_train, y_pred)
        plt.figure(figsize=(6,5))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")
        plt.title("Matriz de confusión - Stroke")
        plt.xlabel("Predicción")
        plt.ylabel("Valor real")
        plt.savefig(os.path.join(figures_dir, "confusion_matrix.png"), bbox_inches="tight")
        plt.close()
    except Exception:
        pass

def split_data() -> None:
    data = read_csv("./healthcare-dataset-stroke-data.csv")

    data["gender"] = data["gender"].astype("category")
    data["ever_married"] = data["ever_married"].astype("category")
    data["work_type"] = data["work_type"].astype("category")
```

Código Python - Stroke (líneas 51-100)

```
data["residence_type"] = data["residence_type"].astype("category")
data["smoking_status"] = data["smoking_status"].astype("category")

data["gender_code"] = data["gender"].cat.codes
data["ever_married_code"] = data["ever_married"].cat.codes
data["work_type_code"] = data["work_type"].cat.codes
data["smoking_status_code"] = data["smoking_status"].cat.codes

# DataFrame({ "Car_Name": data["Car_Name"], "Car_Name_Code": data["Car_Name_Code"]}).to_csv("car_name_and_codes.csv")
# DataFrame({ "Fuel_Type": data["Fuel_Type"], "Fuel_Type_Code": data["Fuel_Type_Code"]}).to_csv("fuel_type_and_codes.csv")
# DataFrame({ "Seller_Type": data["Seller_Type"], "Seller_Type_Code": data["Seller_Type_Code"]}).to_csv("seller_type_and_codes.csv")
# DataFrame({ "Transmission": data["Transmission"], "Transmission_Code": data["Transmission_Code"]}).to_csv("transmission_and_codes.csv")
# exit(0)
train_data: DataFrame
test_data: DataFrame
train_data, test_data = train_test_split(
    data, test_size=0.2, random_state=67
) # type: ignore
train_data.to_csv("train_data.csv")
test_data.to_csv("test_data.csv")

def test_model() -> None:
    base_dir = os.path.dirname(__file__)
    model: RandomForestClassifier = load(os.path.join(base_dir, "stroke.model.pkl"))
    test = read_csv(os.path.join(base_dir, "test_data.csv"))
    y_true = test["stroke"]
    x_test = test[
        [
            "age",
            "hypertension",
            "heart_disease",
            "avg_glucose_level",
            "ever_married_code",
            "bmi",
        ]
    ]
    y_pred = model.predict(x_test)
    print("Accuracy =", accuracy_score(y_true, y_pred))
    from sklearn.metrics import classification_report
    print("Reporte de clasificación:\n", classification_report(y_true, y_pred))

# export figures for test evaluation
figures_dir = os.path.join(base_dir, "figures")
os.makedirs(figures_dir, exist_ok=True)
try:
    report = classification_report(y_true, y_pred, output_dict=True)
    report_df = pd.DataFrame(report).transpose()
    report_df.to_csv(os.path.join(figures_dir, "classification_report_test.csv"))
```

Código Python - Stroke (líneas 101-129)

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")
plt.title("Matriz de confusión - Stroke (test)")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.savefig(os.path.join(figures_dir, "confusion_matrix_test.png"), bbox_inches="tight")
plt.close()
except Exception:
    pass

if __name__ == "__main__":
    test_model()
    # split_data()
    exit(0)
    train_data = read_csv("./train_data.csv")
    y_train = train_data["stroke"]
    x_train = train_data[
        [
            "age",
            "hypertension",
            "heart_disease",
            "avg_glucose_level",
            "ever_married_code",
            "bmi",
        ]
    ]
    build_model(x_train, y_train)
```